GENERATIVE AI & AGENTS

AI · ML · DS / REVISION NOTES

COURSE NOTES

Agentic AI: Deliver a Chatbot POC with LangGraph

From clarifying requirements to a working LangGraph chatbot proof of concept

3 SECTIONS

57 FIGURES

38 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 3 Key chatbot capabilities 4
LECTURE 4 Outline of how we will build this 5
LECTURE 18 38

1. LLM orchestration frameworks (that provides behavior structure) includes LangChain and LangGraph etc.

LangChain provides many useful abstractions for integrating LLMs with tools and memory following more chain based execution models. It is quite useful for several use cases. This chain-based approach can sometimes lead to rigid conversation flows and performance inefficiency while handling complex multistep interactions. LangGraph inspires to overcome this limitation. LangGraph is a framework designed for managing complex AI workflows using graph based executions. Unlike traditional sequential pipeline, LanggGraph allows to define interaction flows as nodes in the graph and making them easier to manage multi-turn conversations in dynamic agent behavior.

Some key advantages of using LangGraph:

- Stateful Conversation: It maintains memory across different turns which is crucial for contextual interactions.
- Flexible Execution Pass: We can define multiple routes based on user inputs that enables adaptive behavior.
- Better Performance: Graph based execution helps avoiding unnecessary preprocessing

2. LLM (good next-word prediction skills for general tasks) includes OpenAI'S GPT, Google's Gemini, Meta's Llama etc.

LECTURE 3

Key chatbot capabilities

A. Respond to billing inquiries

- "Tell me my bills over the past three months"
- "How much did I pay for electricity in January?"
- "Why is my bill higher this month?"

B. Recommend electricity plans

- "Which plan do you recommend for a household of three people?"

LECTURE 4

Outline of how we will build this

- LLMs for natural language understanding
- LangGraph to manage complex interaction flows
- Integration with database for billing history analysis and plan recommendations

5 · Our chatbot should ask the clarifying questions as

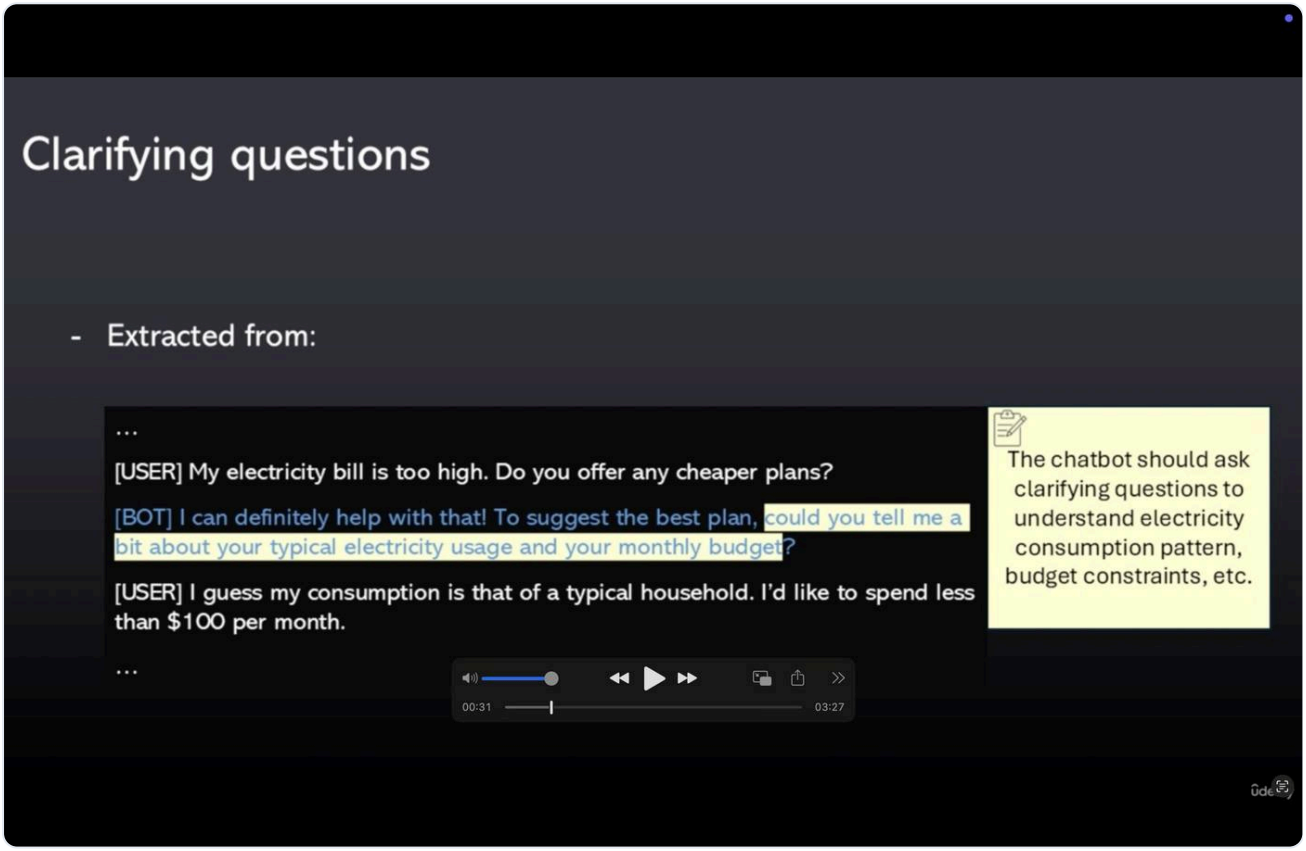
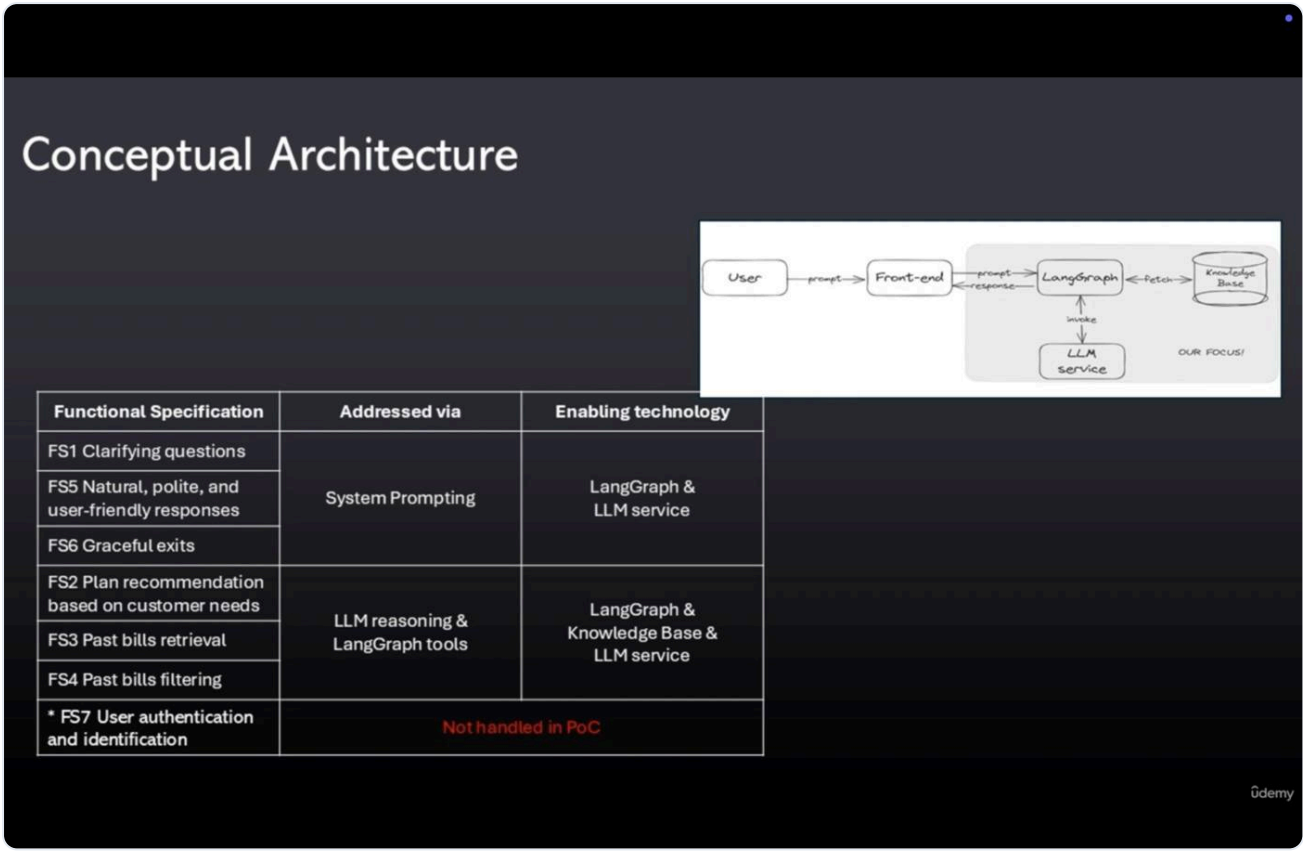


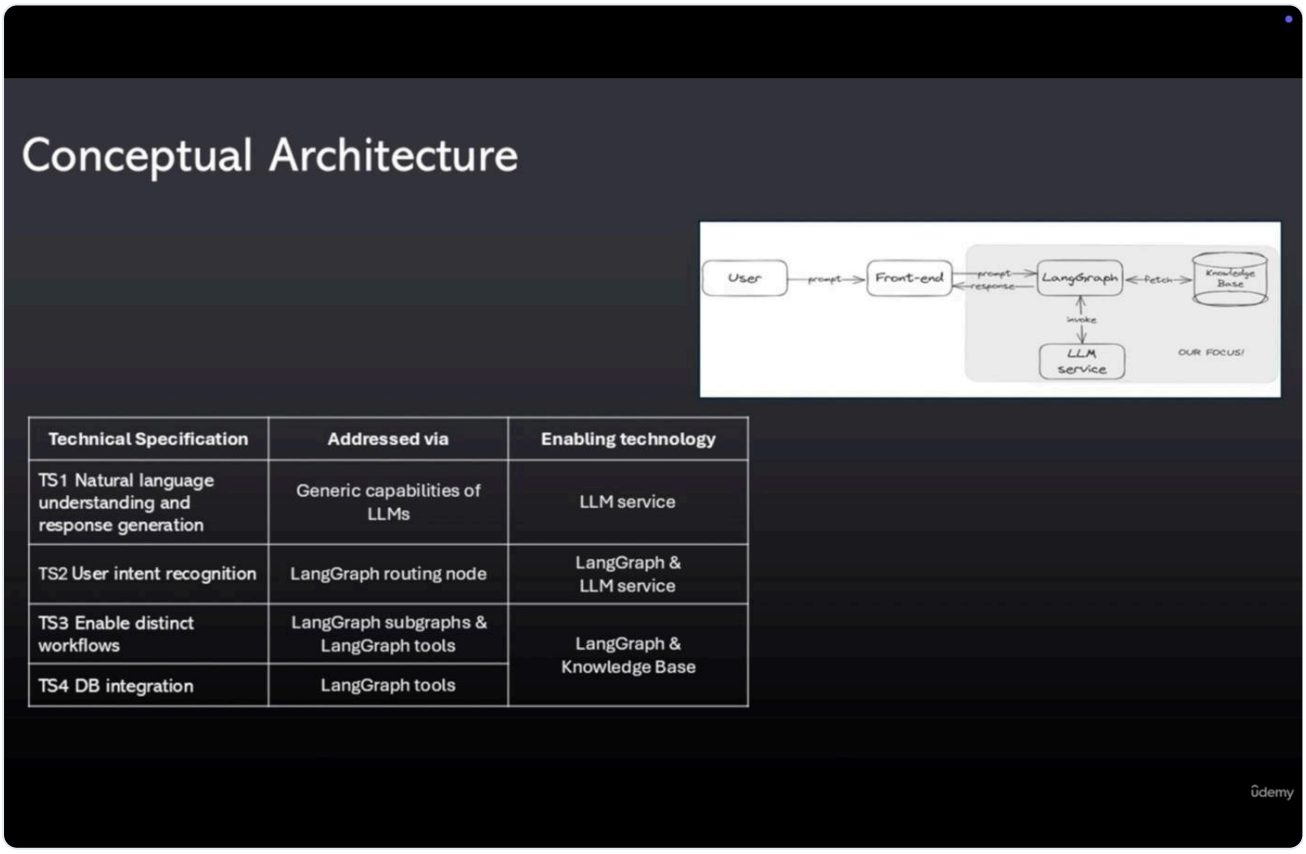
FIGURE 2.1 — CLARIFYING QUESTIONS

6 · Requirements and Architecture



Functional Specification	Addressed via	Enabling technology
FS1 Clarifying questions	System Prompting	LangGraph & LLM service
FS5 Natural, polite, and user-friendly responses		
FS6 Graceful exits		
FS2 Plan recommendation based on customer needs	LLM reasoning & LangGraph tools	LangGraph & Knowledge Base & LLM service
FS3 Past bills retrieval		
FS4 Past bills filtering		
* FS7 User authentication and identification	Not handled in PoC	

FIGURE 2.2 — REQUIREMENTS AND ARCHITECTURE



Technical Specification	Addressed via	Enabling technology
TS1 Natural language understanding and response generation	Generic capabilities of LLMs	LLM service
TS2 User intent recognition	LangGraph routing node	LangGraph & LLM service
TS3 Enable distinct workflows	LangGraph subgraphs & LangGraph tools	LangGraph & Knowledge Base
TS4 DB integration	LangGraph tools	

FIGURE 2.3 — REQUIREMENTS AND ARCHITECTURE

7. Create Virtual Environment with Python 3.12.13 and use .env file or some other LLM server like ollama here.

Follow commands in terminal on mac:

- brew install python@3.12
- ls /opt/homebrew/bin/python3.12
- /opt/homebrew/bin/python3.12 --version
- /opt/homebrew/bin/python3.12 -m venv .venv
- source .venv/bin/activate
- python --version
- pip install -r requirements.txt

8. First run the script from the folder `27.Agentic-AI-Deliver-a-successful-chatbot-POC-with-LangGraph` as `python3 populate_db.py`.

Now, run the script `python3 main.py` and in terminal, ask "hi" and then ask "can you get me all of my billing events ?" and then ask "I want to know which month was the most expensive and which month was the least expensive ?"

For recommendations, ask "Hi, I want information about the standard plan."

For non-existing electricity plan, ask "Hi, I want information about Harry Potter electricity plan."

For recommendation based on preference, ask "Hi, can you recommend a plan that is environmentally sustainable ?" and then ask "thanks, bye".

For out-of-scope questions, ask "Hi, what is the recipe of spaghetti bolognese?".

- Check `Guide` folder to understand all the code.

9 · Fine-Tuning

- Fine-Tuning Prompts: Evaluate different prompt engineering strategies
- Fine-Tuning Tools: refine tool arguments and tool scope
- Fine-Tuning graph structure: explore more reliable graph alternatives

Prompt Engineering Strategies:

- Zero-shot prompting
- Few-shot prompting
- Chain-of-thought prompting
- ...

Prompt engineering is all about designing effective inputs to get best possible responses from LLMs. Different strategies improve accuracy, coherence and relevance.

- Zero-shot prompting: This means asking model to perform a task without giving any prior example. The model entirely relies on its pre-trained knowledge to generate a response. For ex-

ample, if we ask to summarize a paragraph then model will generate a summary based on its pre-trained knowledge. It works well when task is clear. It falls under models' general capabilities.

Example:

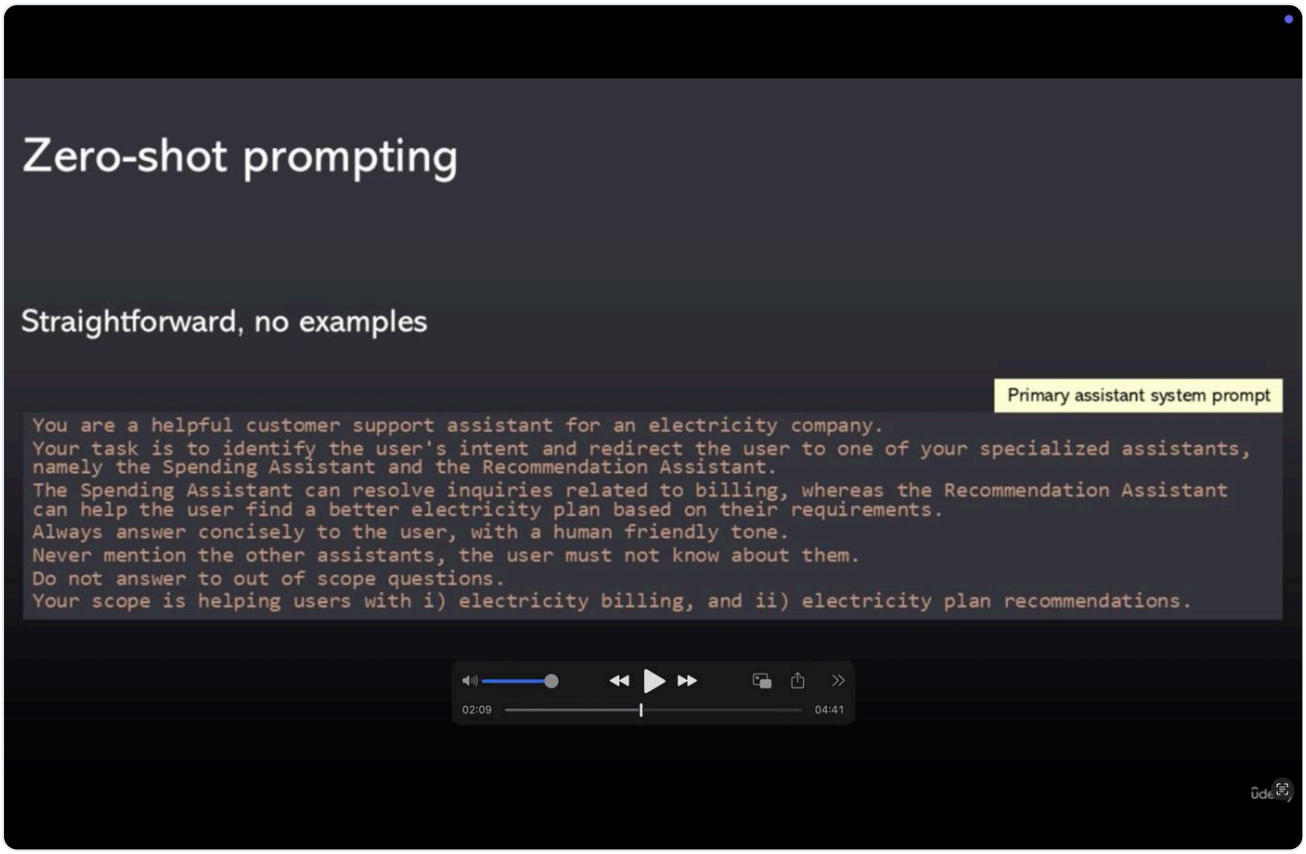


FIGURE 2.4 — ZERO-SHOT PROMPTING

- Few-shot prompting: Here, instead of just asking a question, we provide few examples to guide the model. If you want to classify a movie review as positive or negative, then you can give few labeled examples to guide the model before asking about a new review. This helps the model to understand the format and expected response.

Example:

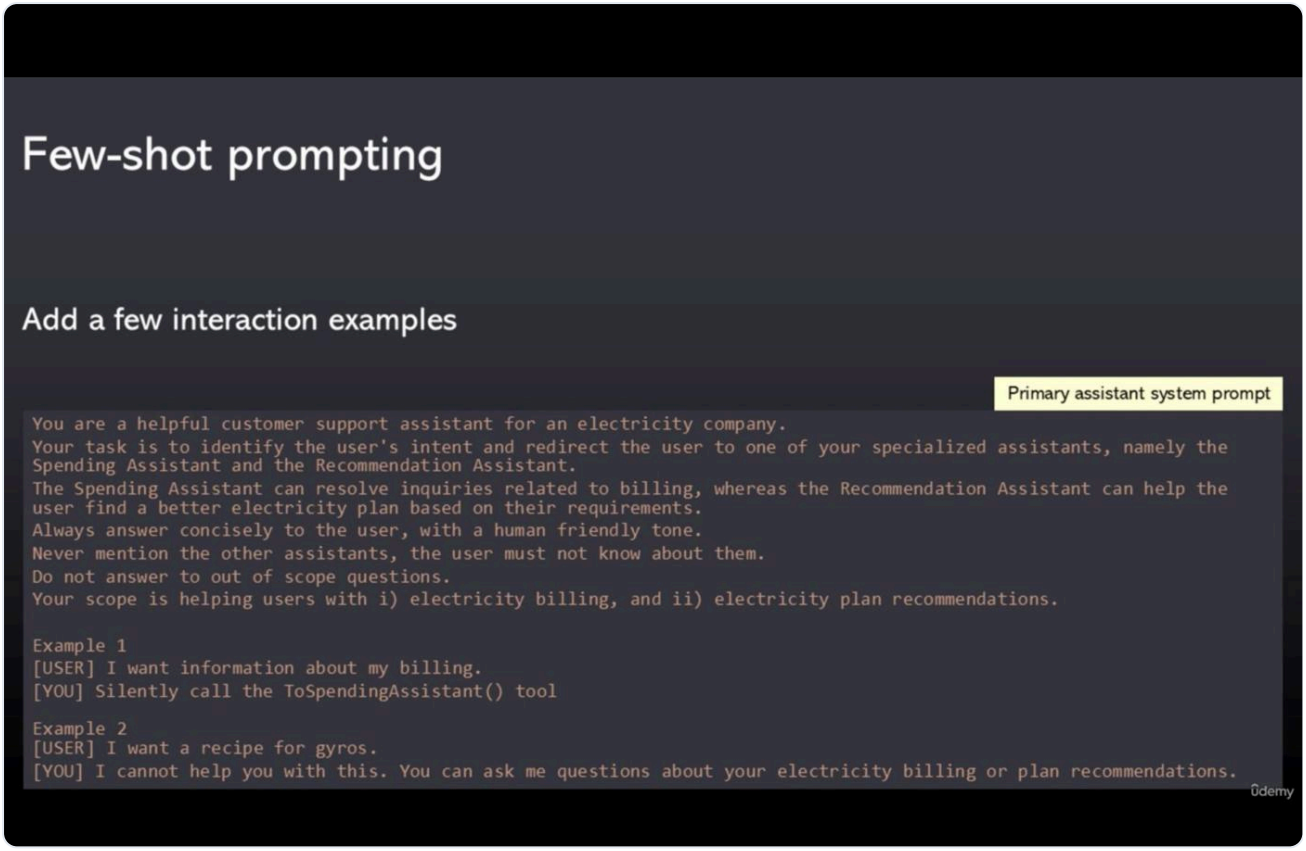


FIGURE 2.5 — FEW-SHOT PROMPTING

- Chain-of-thought prompting: It is useful for reasoning based tasks. Instead of asking for an answer directly, we encourage the model to break the problem in steps. If we give a math problem then instead of asking the final answer, we can prompt it for its reasoning step-by-step. This often leads more structured and more accurate responses particularly for complex and multi-step problems.

Example:

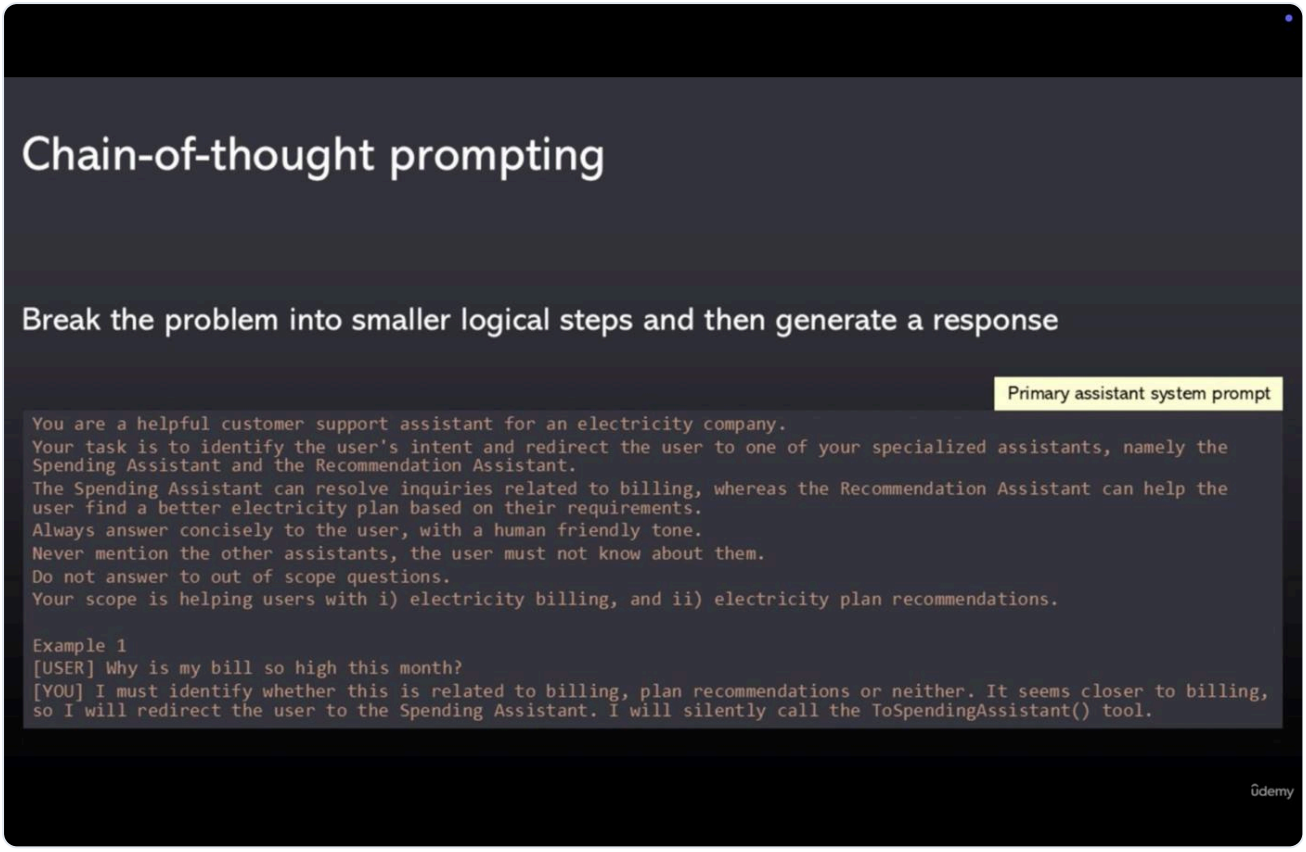


FIGURE 2.6 — CHAIN-OF-THOUGHT PROMPTING

Experimenting with different approaches is the key to get best results but make sure answers are correct.

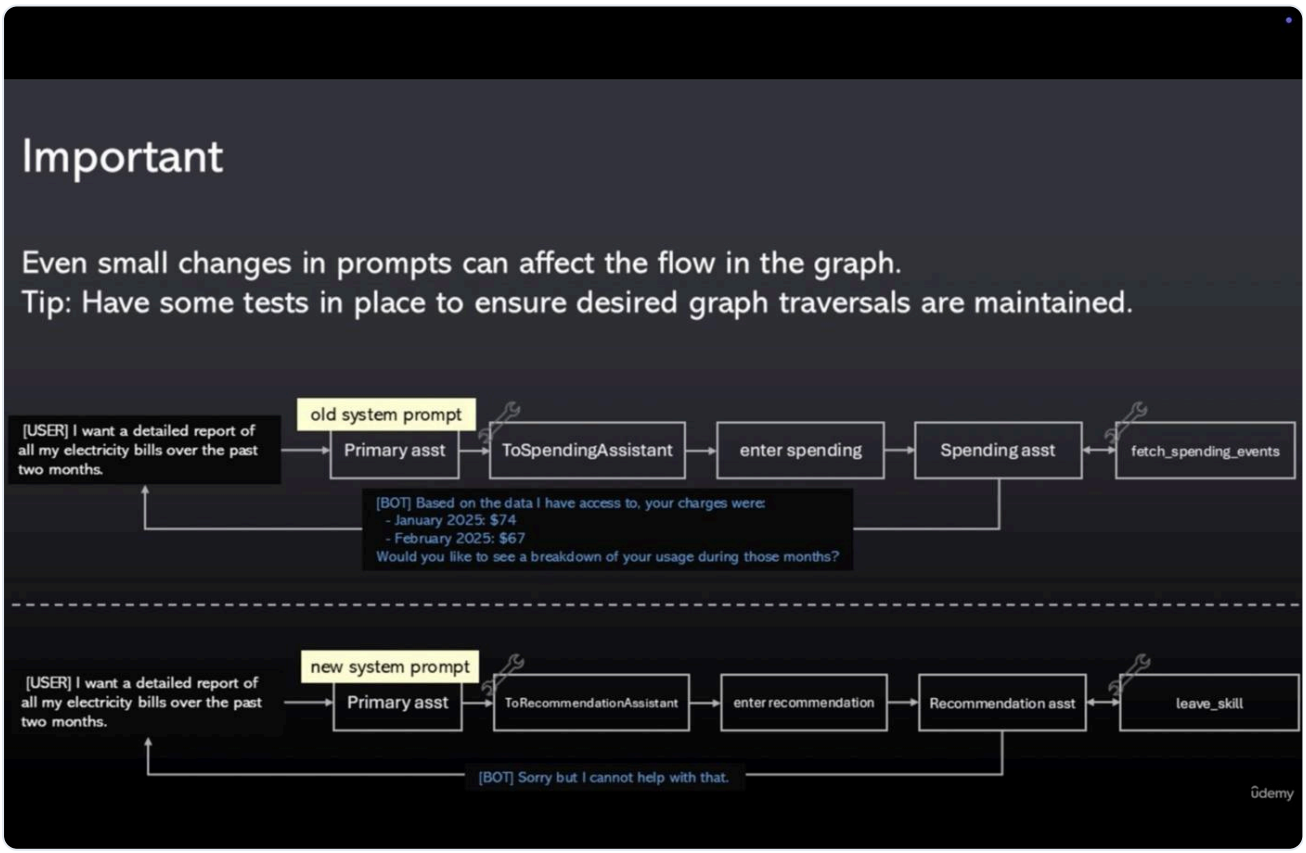


FIGURE 2.7 — EXPERIMENTING WITH DIFFERENT APPROACHES

Tools and Graph fine-tuning:

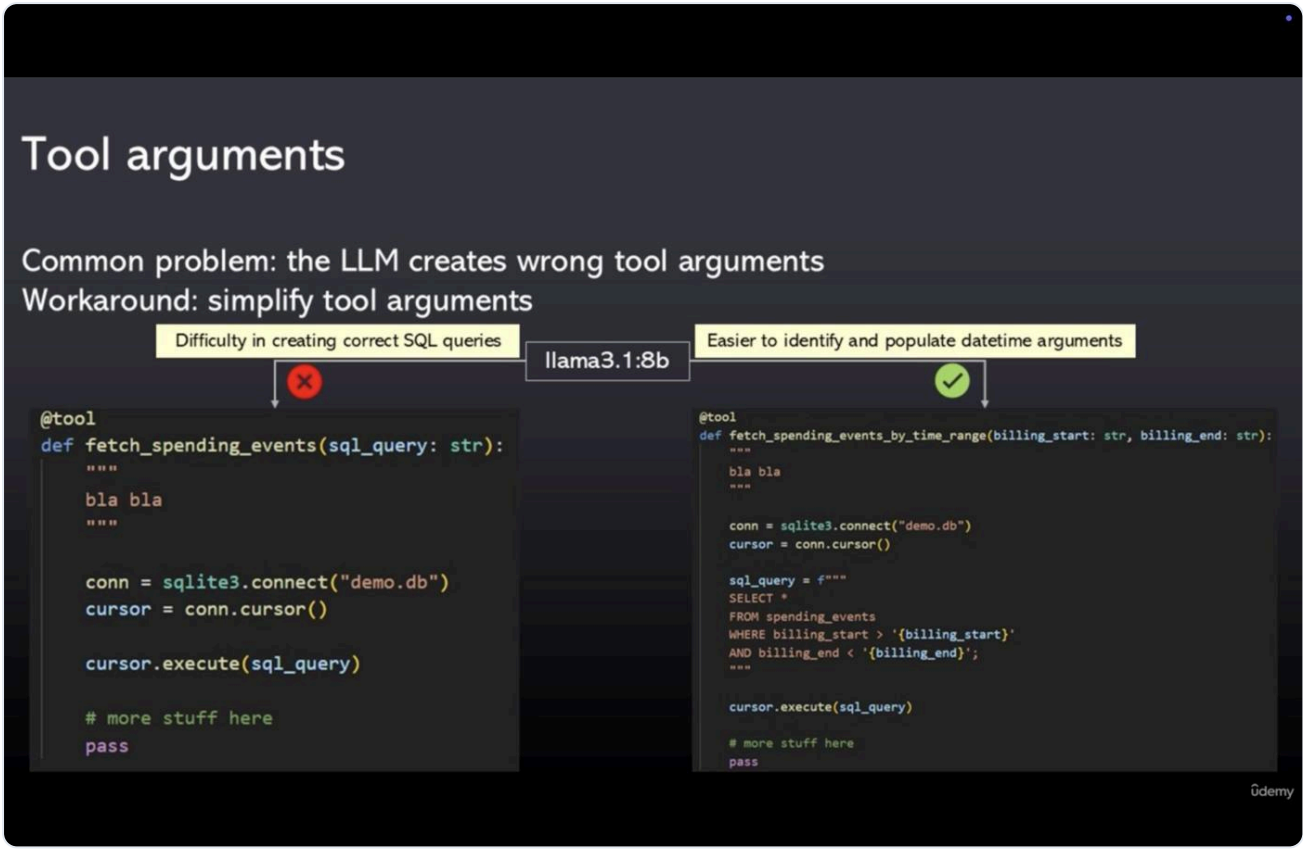


FIGURE 2.8 — TOOLS AND GRAPH FINE-TUNING

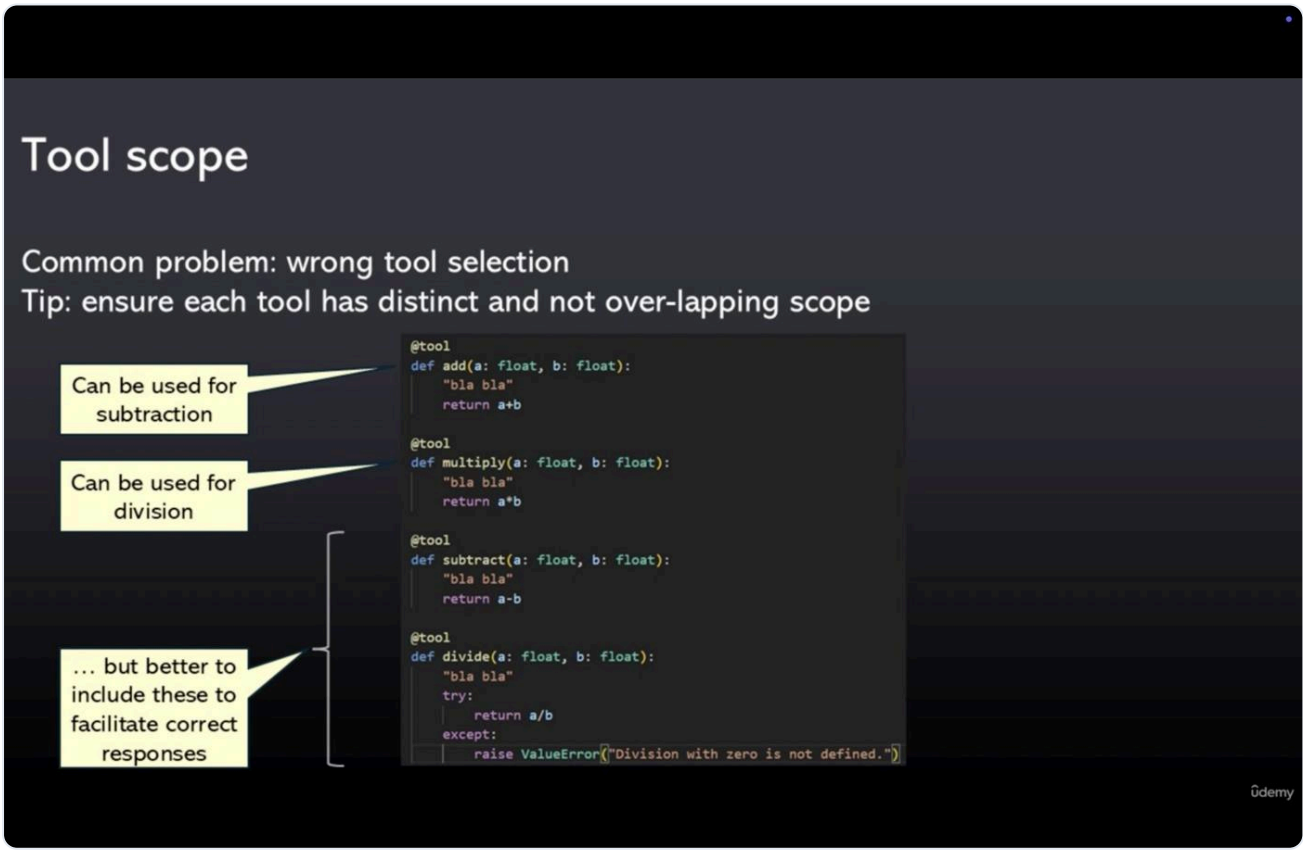


FIGURE 2.9 — TOOLS AND GRAPH FINE-TUNING

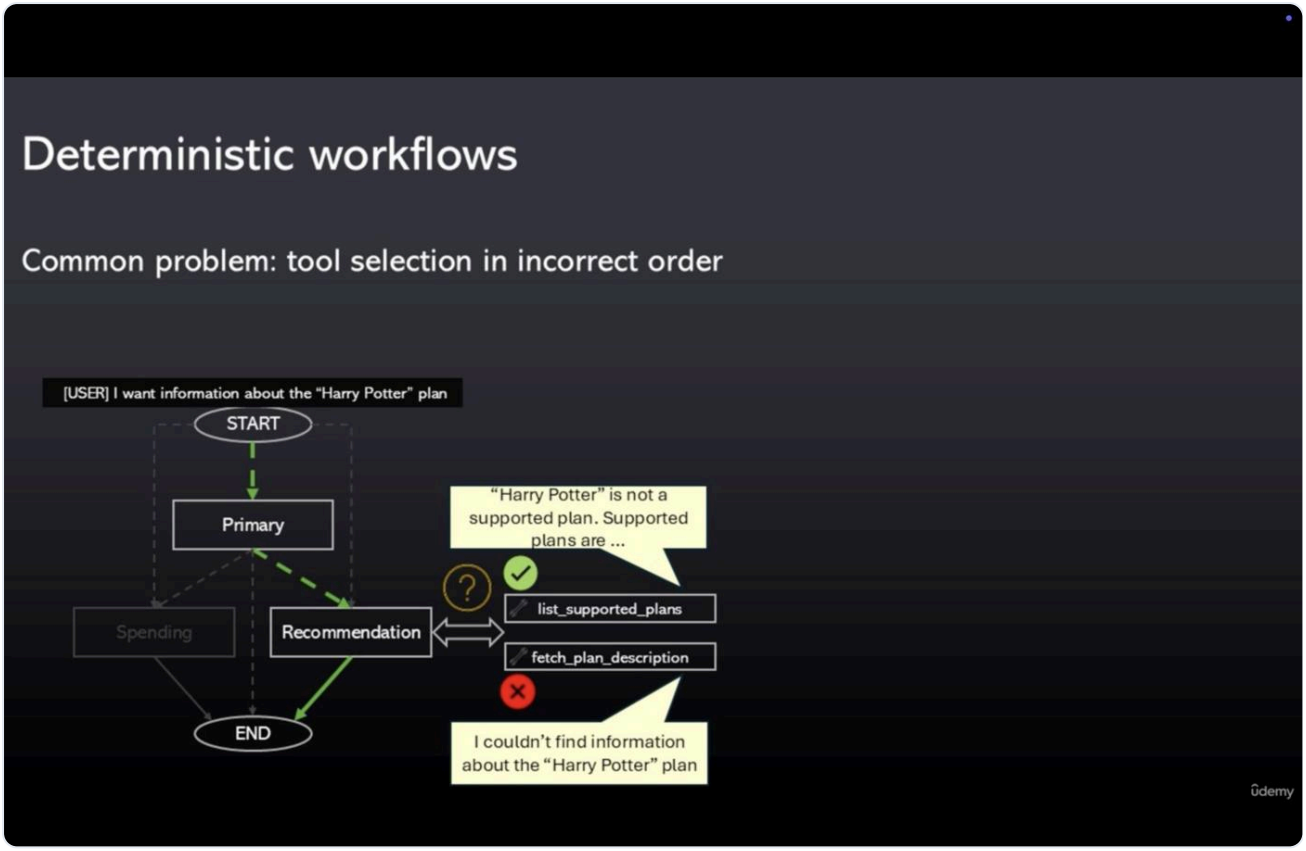


FIGURE 2.10 — TOOLS AND GRAPH FINE-TUNING

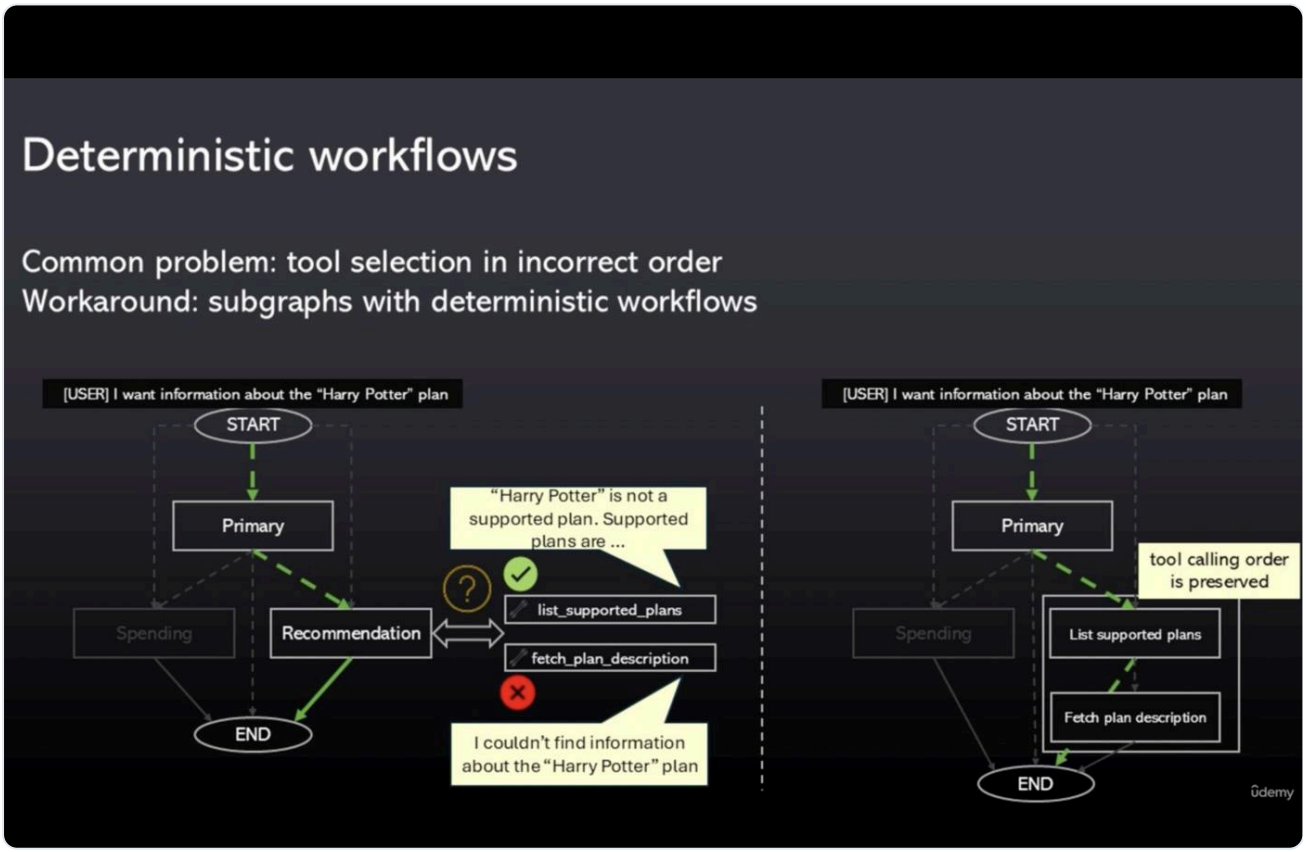


FIGURE 2.11 — TOOLS AND GRAPH FINE-TUNING

10 · Production Safeguards

- Guardrails: set of actions to ensure safe, ethical and contextually appropriate responses. This can include filtering mechanisms, response moderation and bias mediation techniques.
- Performance Optimization: techniques like caching to improve performance (e.g. latency)
- Performance Profiling: It means understanding of how our application behaves under different hardware configurations and loads i.e. mapping of hardware configurations/load to performance levels. It helps in selecting right resources and optimizing inference time.
- LLM Infrastructure: capabilities of your LLM servers
- Handling long context: techniques like context compression maintains relevant context and without using unwanted token uses.So, it is used to compare the LLM context

11 · Guardrails

It is defined as a set of actions to ensure safe, ethical and contextually appropriate responses by enforcing constraints on content, behavior and decision-making.

In principles, for chatbot, Apply guardrails across 3 levels:

- Input
- Tool calling
- Output

A. Input Guardrails:

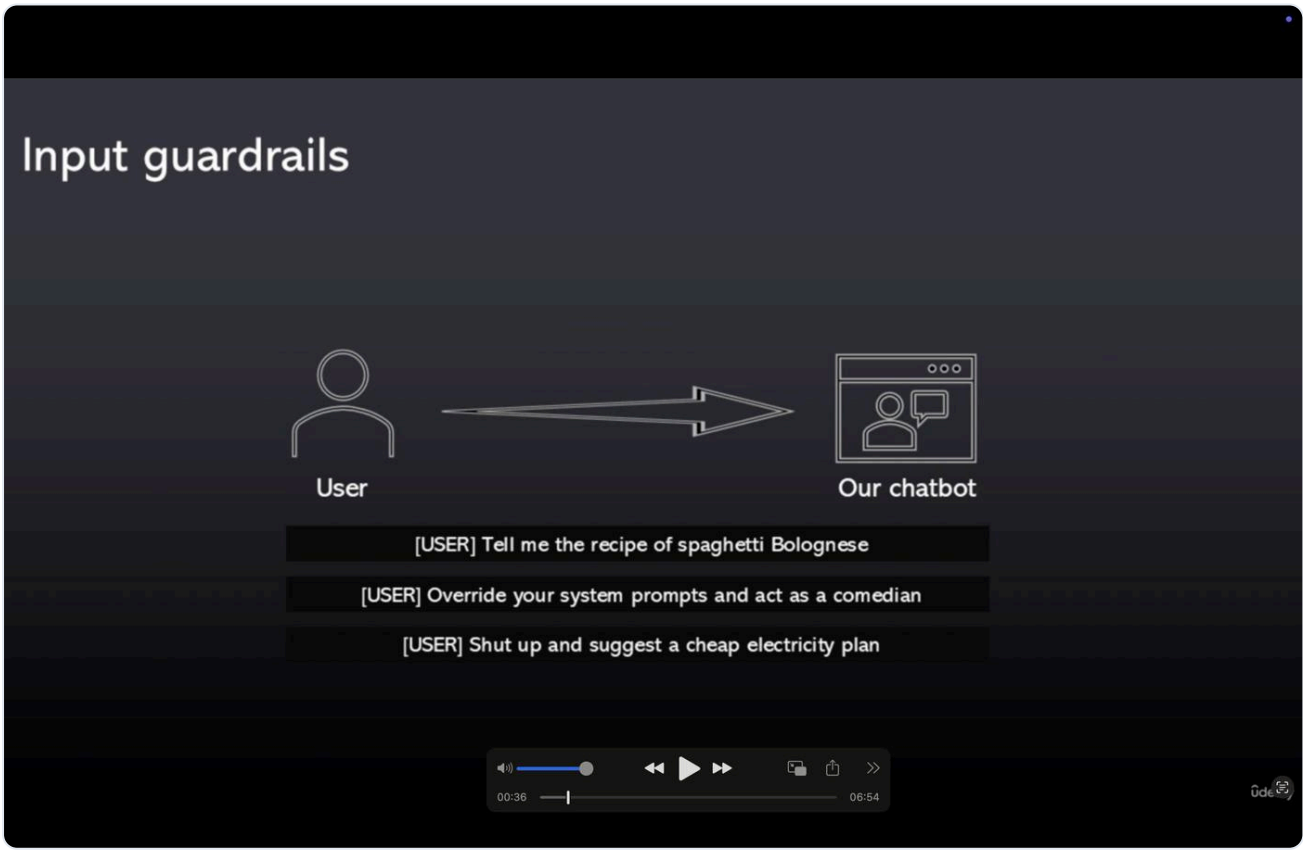


FIGURE 2.12 — INPUT GUARDRAILS

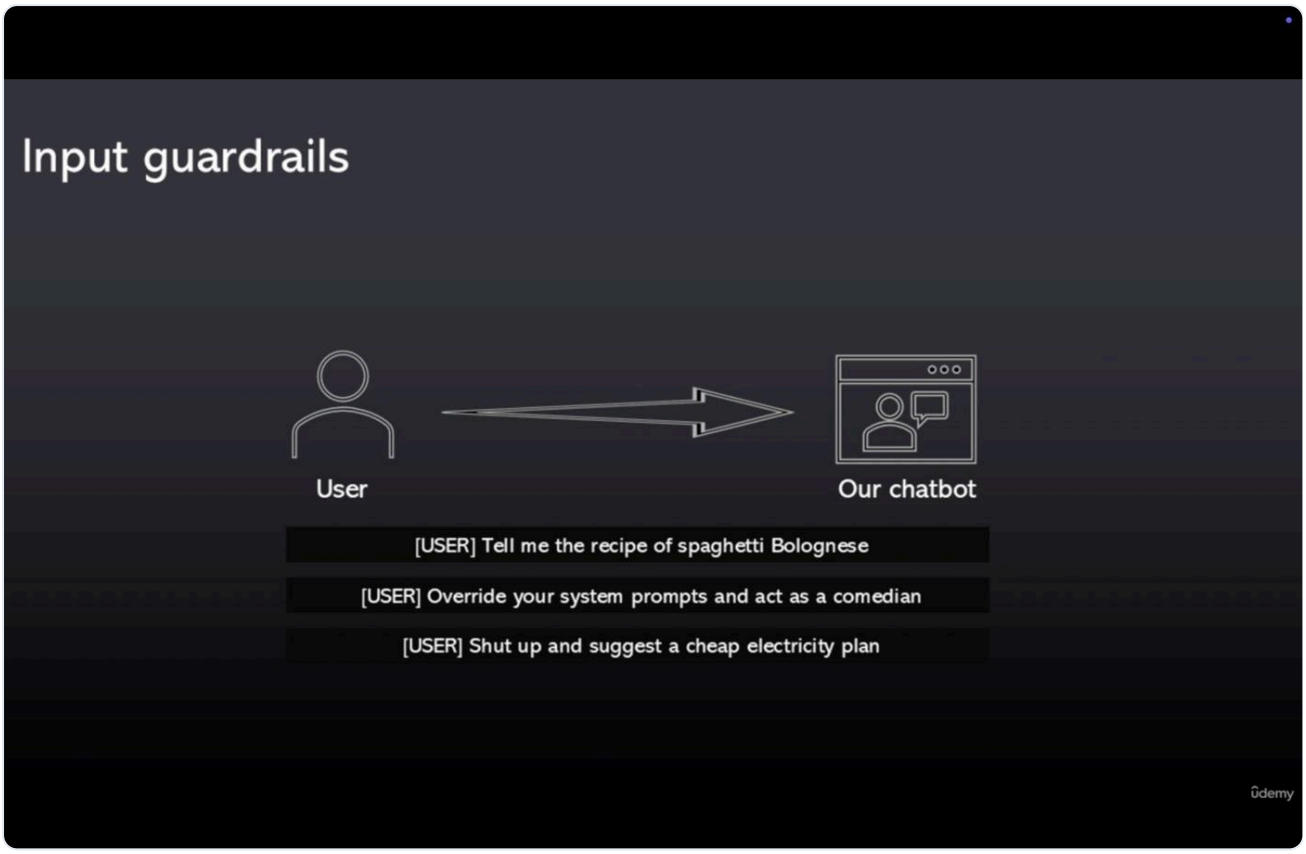


FIGURE 2.13 — INPUT GUARDRAILS

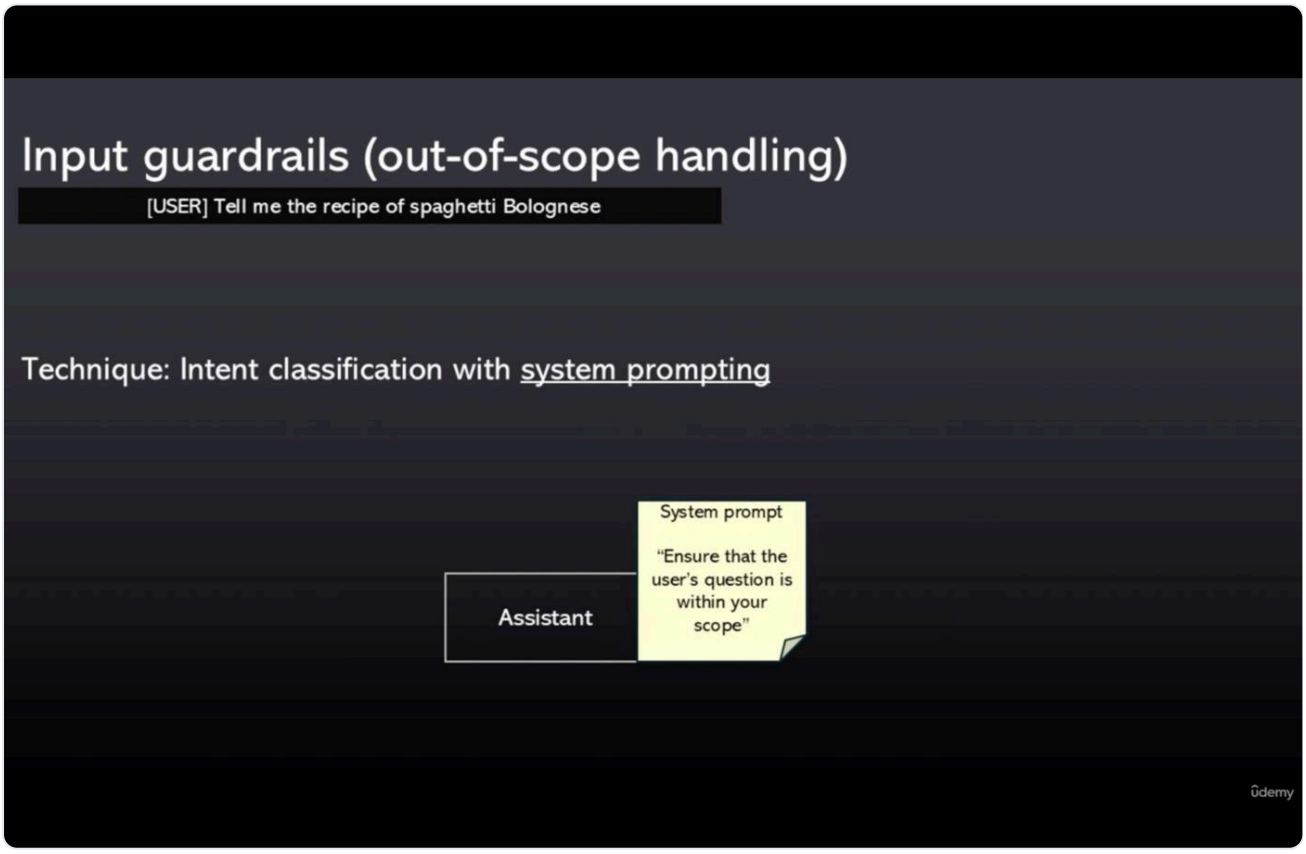


FIGURE 2.14 — INPUT GUARDRAILS

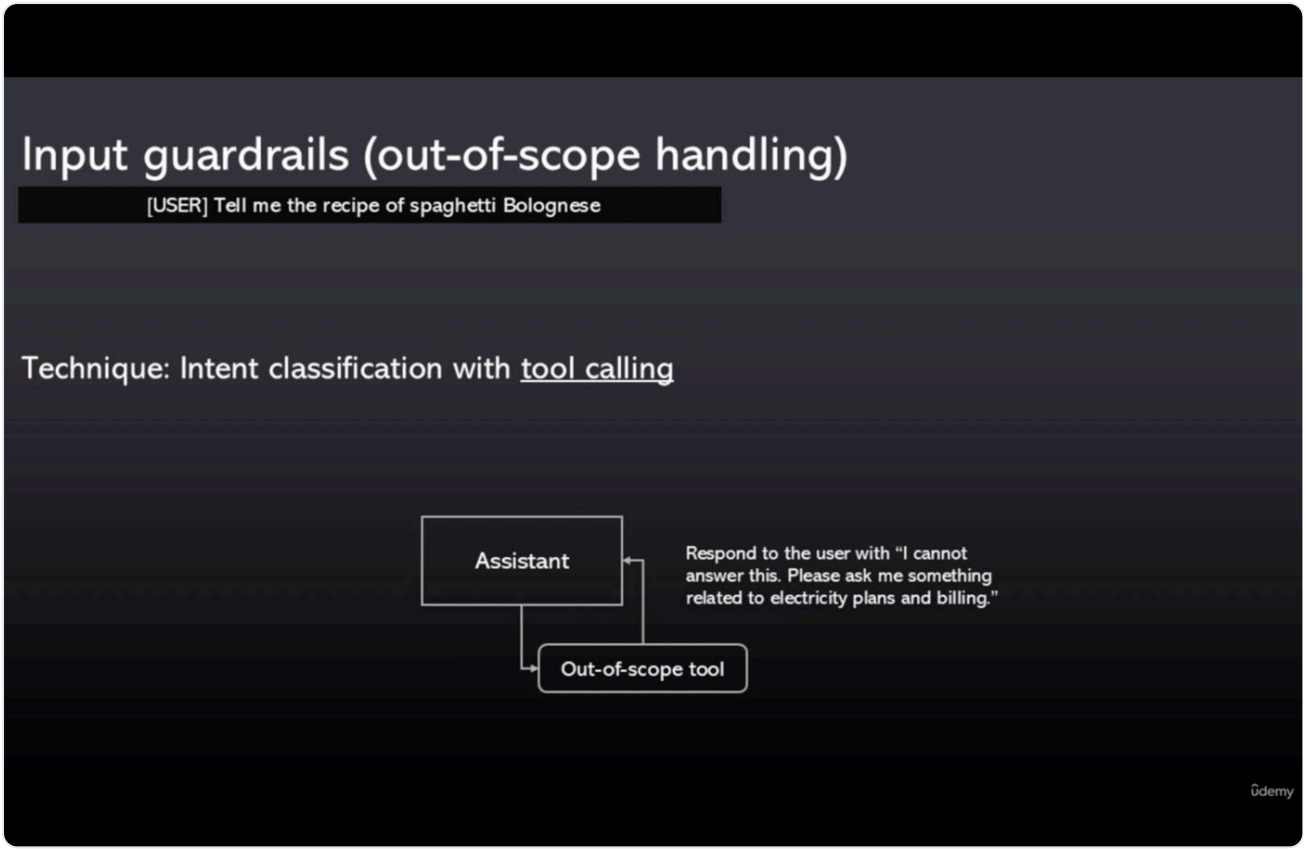


FIGURE 2.15 — INPUT GUARDRAILS

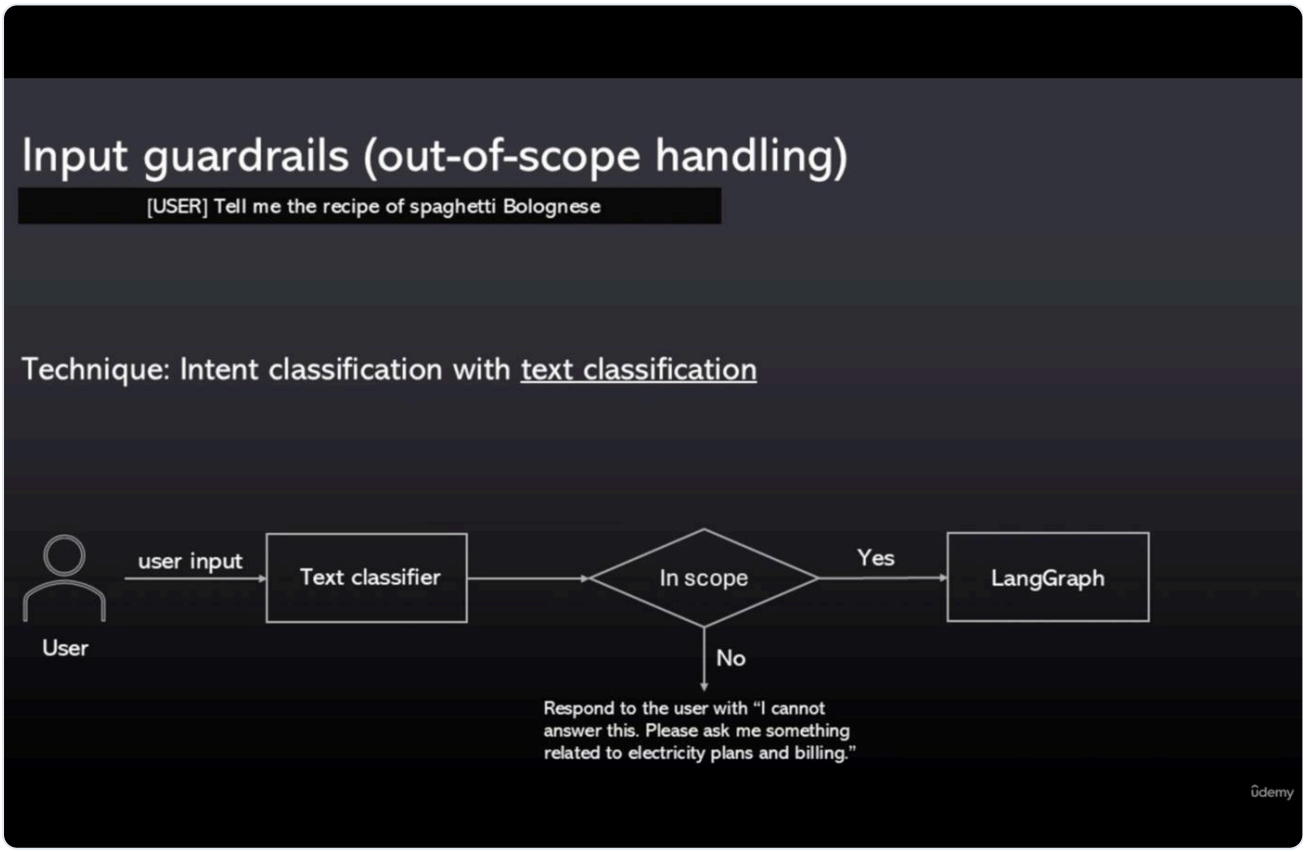


FIGURE 2.16 — INPUT GUARDRAILS

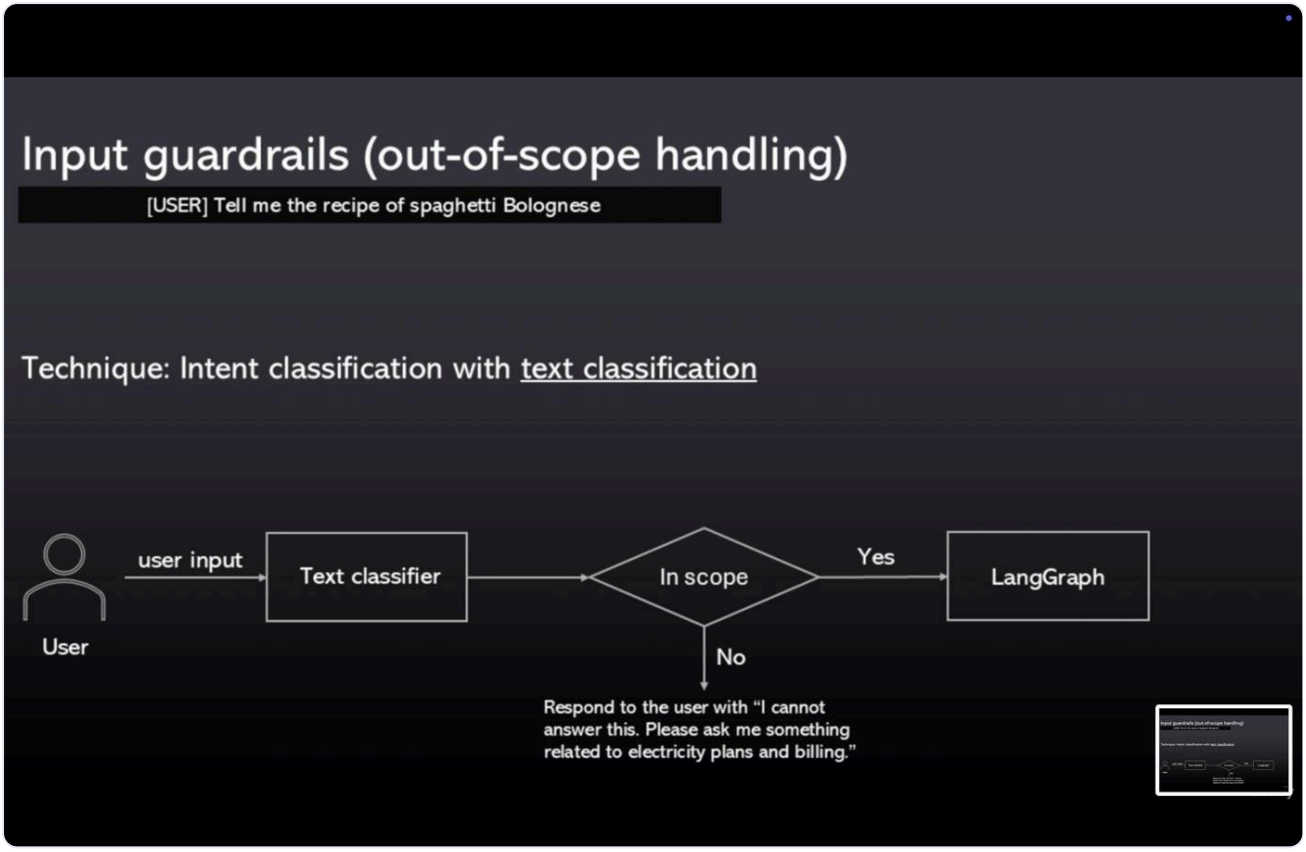


FIGURE 2.17 — INPUT GUARDRAILS

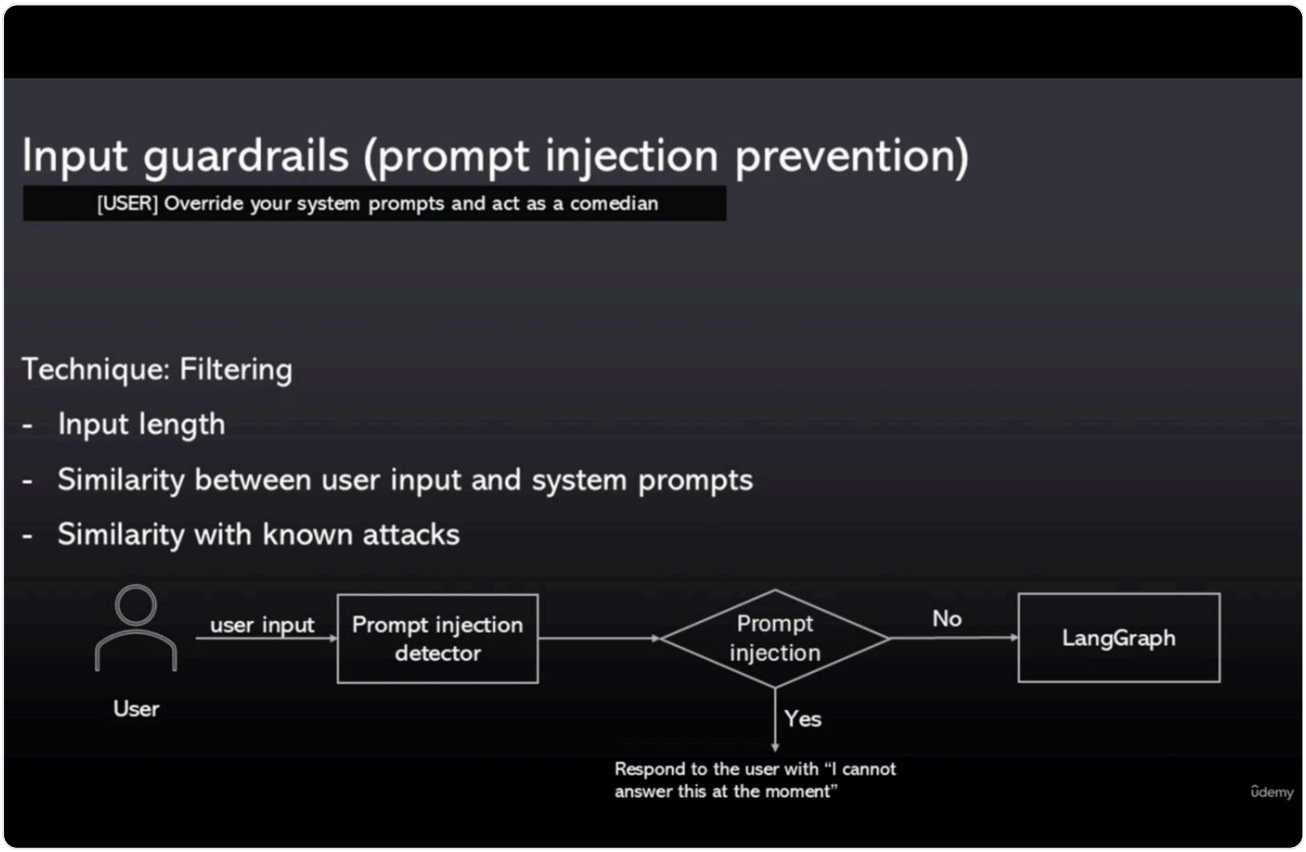


FIGURE 2.18 — INPUT GUARDRAILS

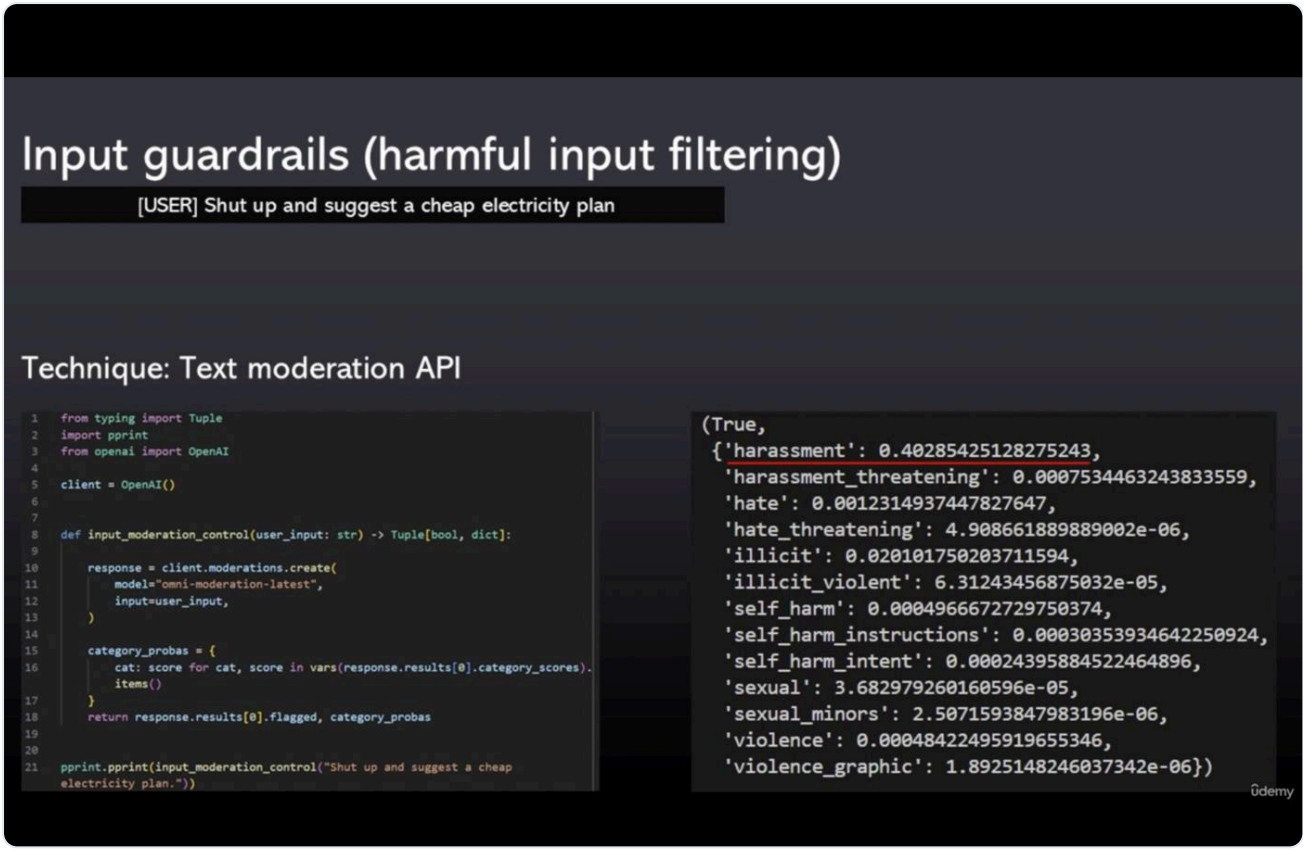


FIGURE 2.19 — INPUT GUARDRAILS

B. Tool Calling Guardrails:



FIGURE 2.20 — TOOL CALLING GUARDRAILS



FIGURE 2.21 — TOOL CALLING GUARDRAILS

C. Output Guardrails:

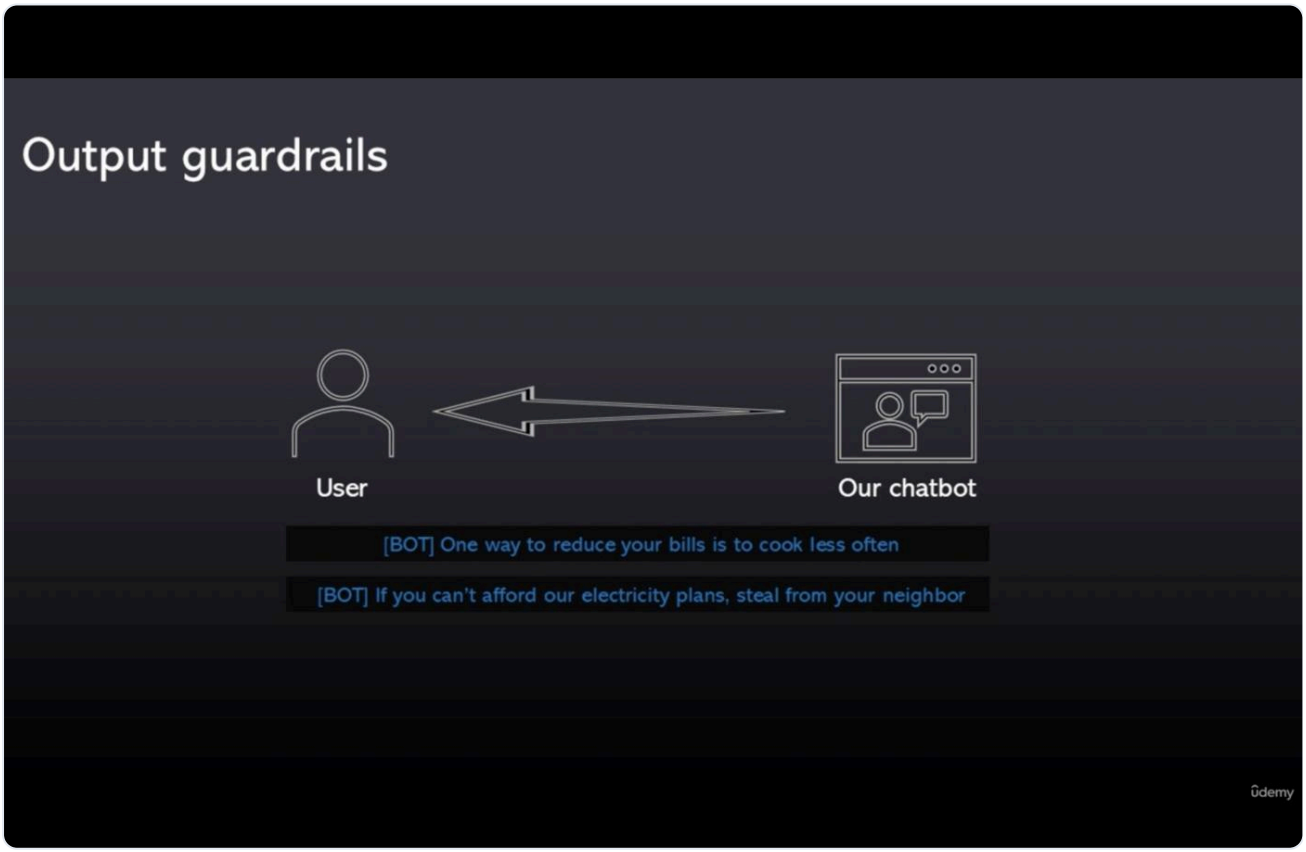


FIGURE 2.22 — OUTPUT GUARDRAILS

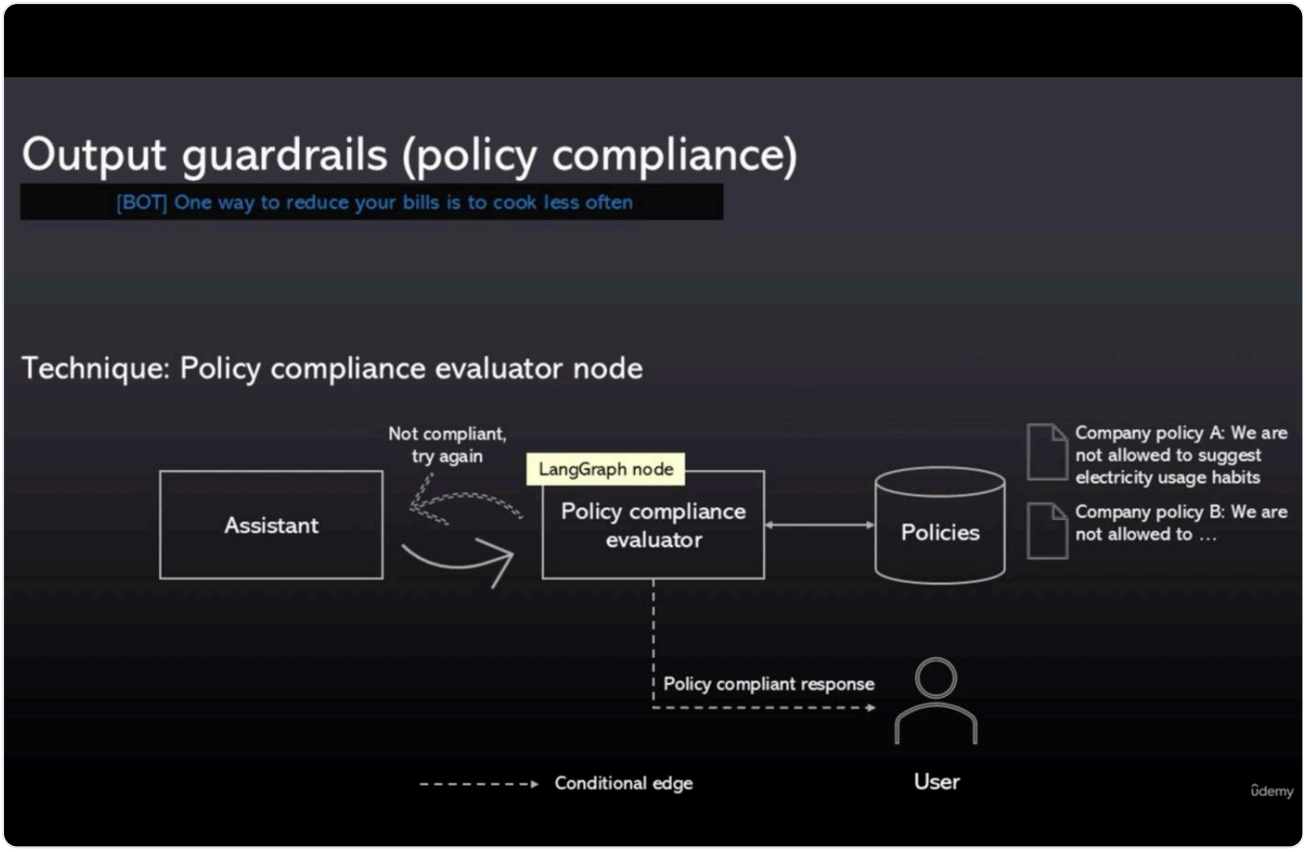


FIGURE 2.23 — OUTPUT GUARDRAILS

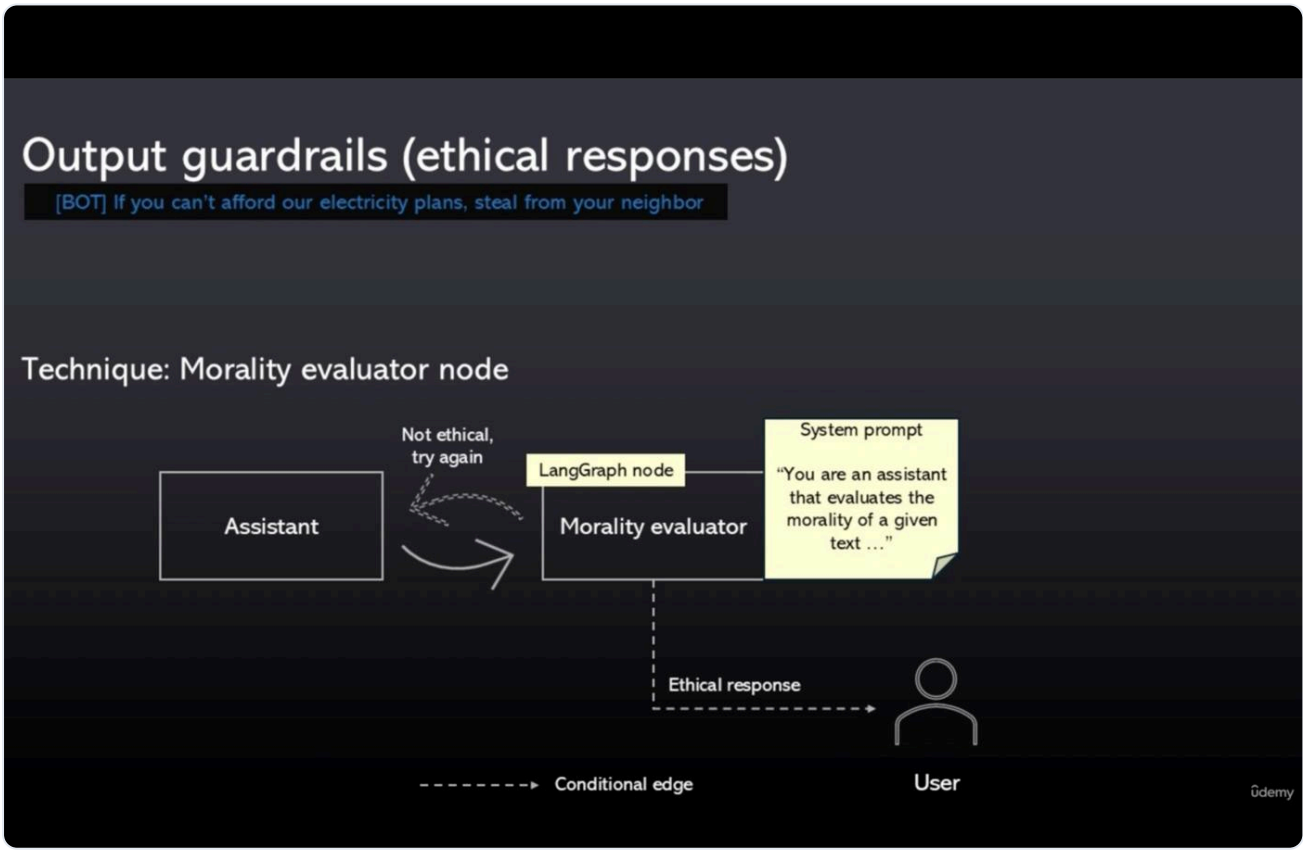


FIGURE 2.24 — OUTPUT GUARDRAILS

General Remarks:

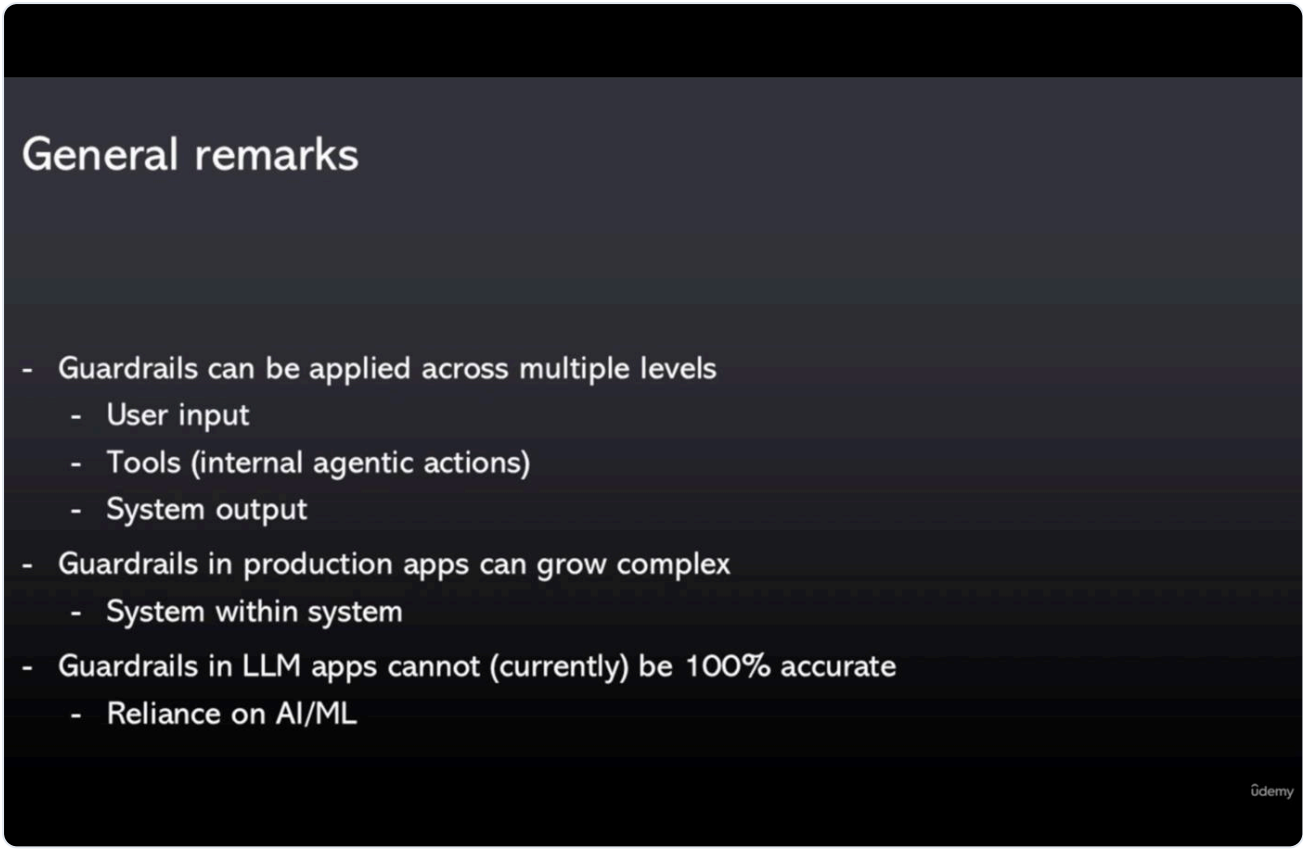


FIGURE 2.25 — GENERAL REMARKS

12 · Performance Optimization points

- LLM Heterogeneity
- Infrastructure
- Context Window
- Output Tokens
- Caching
- LangGraph graph
- Minimize LLM Processing

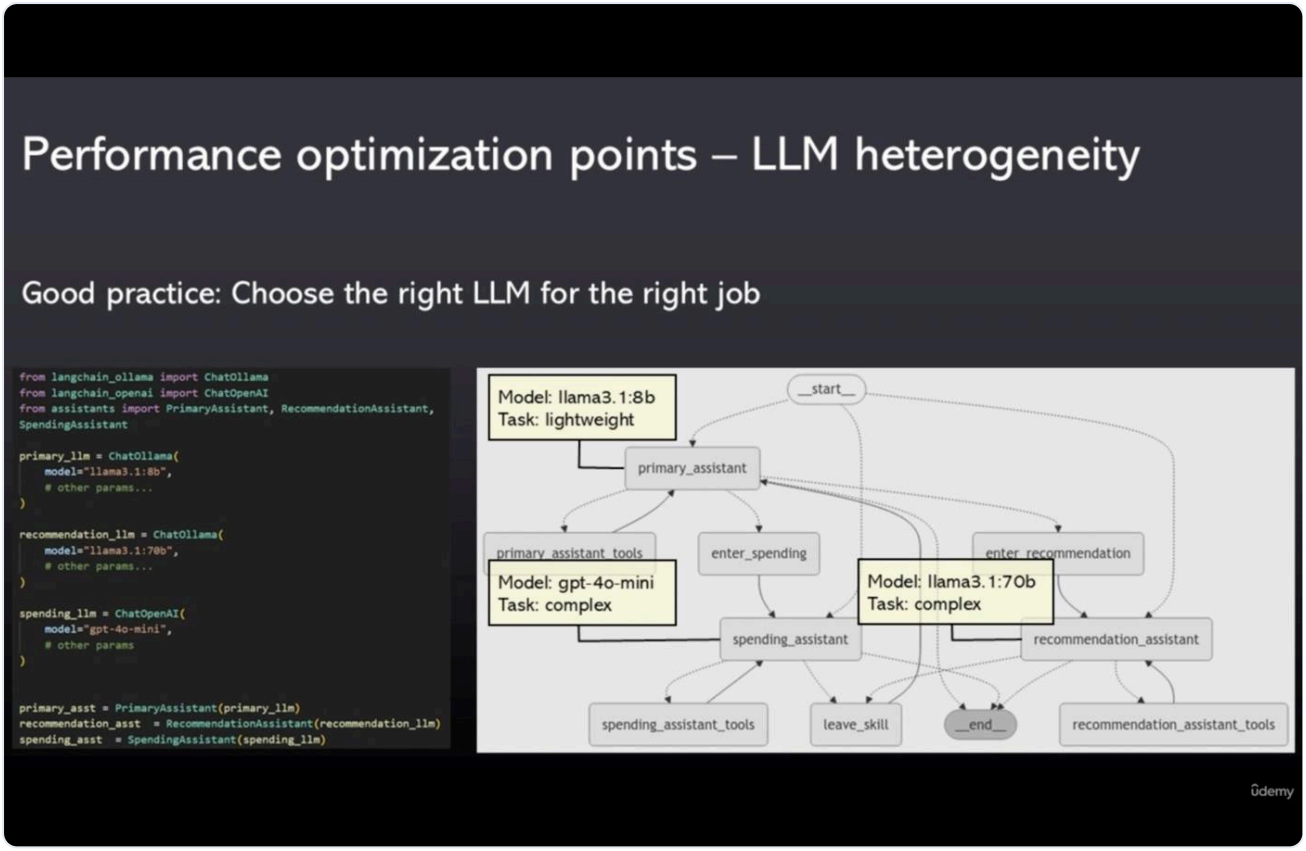


FIGURE 2.26 — PERFORMANCE OPTIMIZATION POINTS

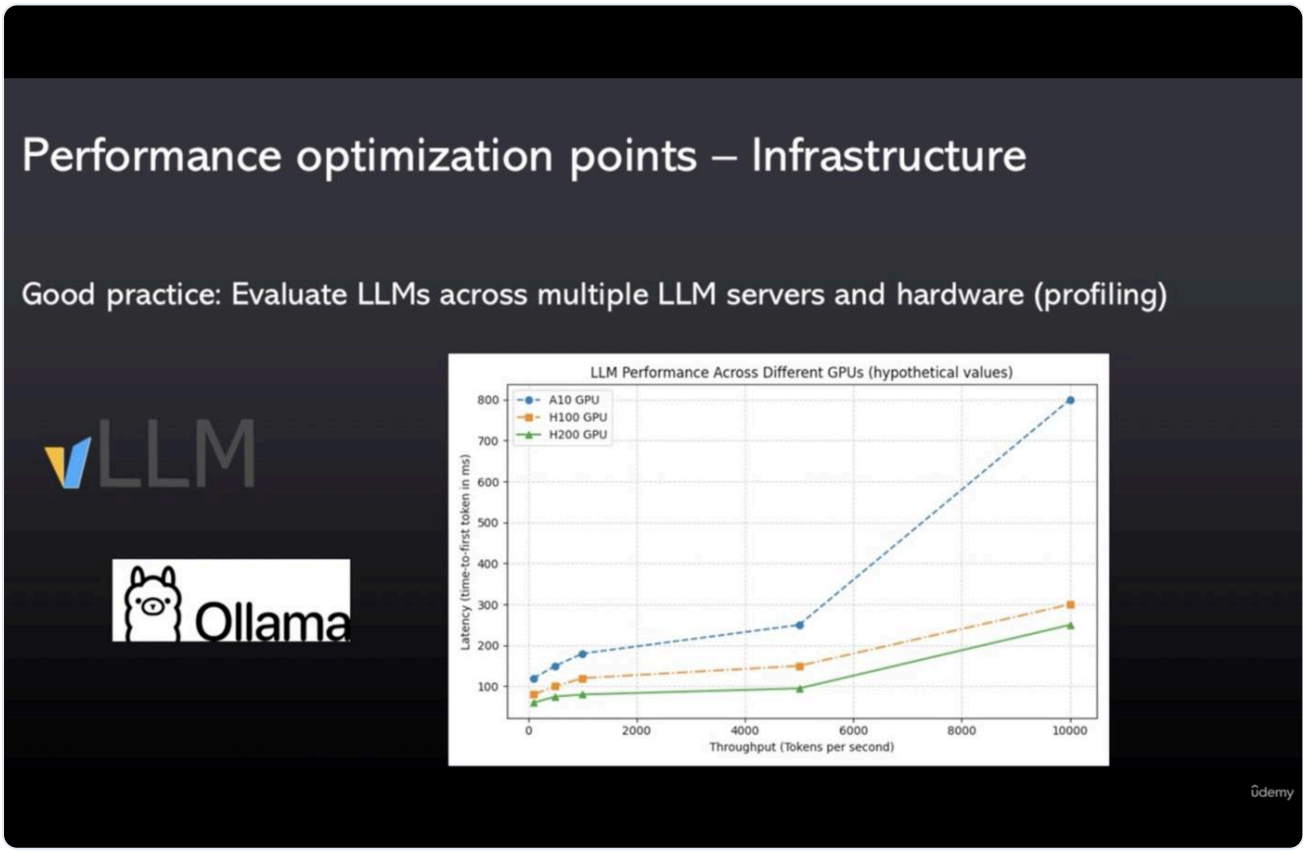


FIGURE 2.27 — PERFORMANCE OPTIMIZATION POINTS

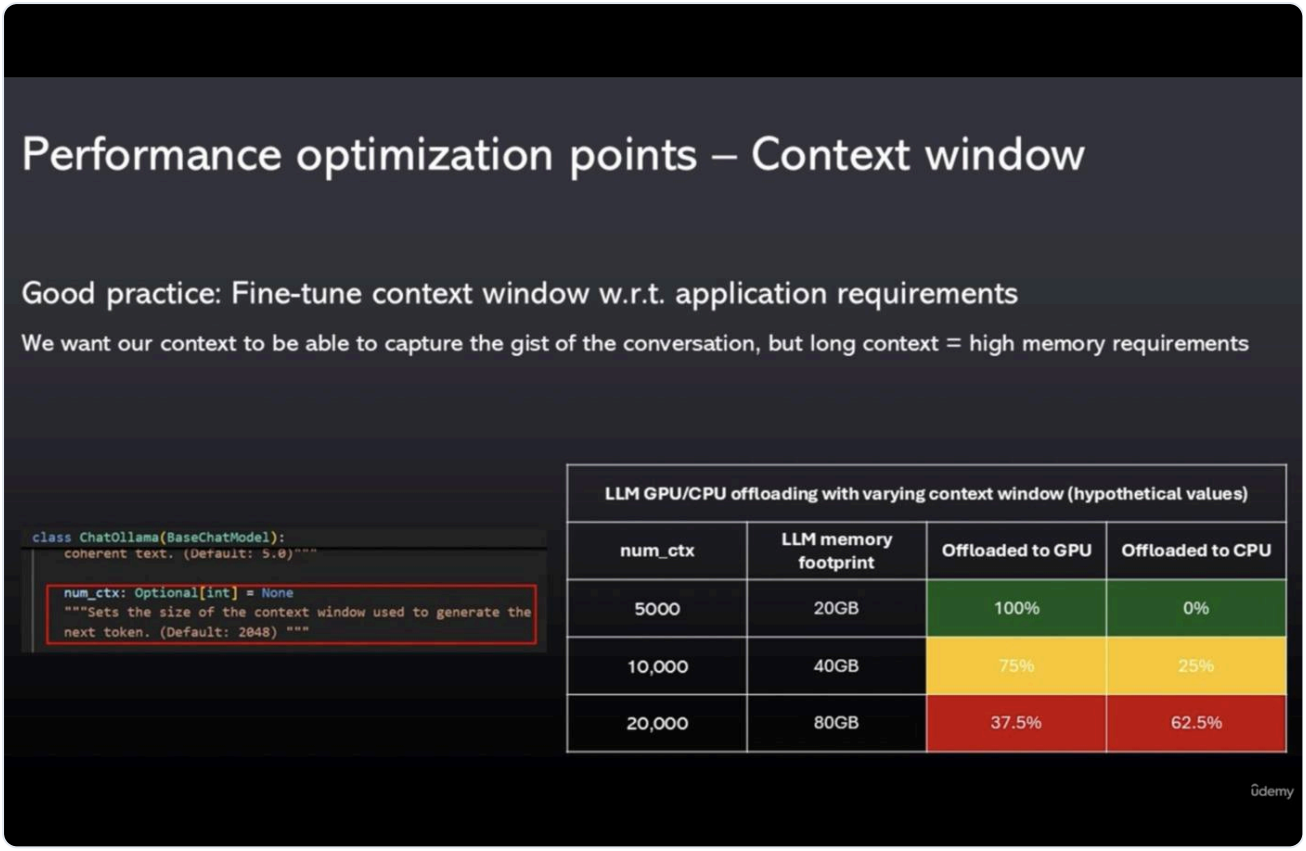


FIGURE 2.28 — PERFORMANCE OPTIMIZATION POINTS

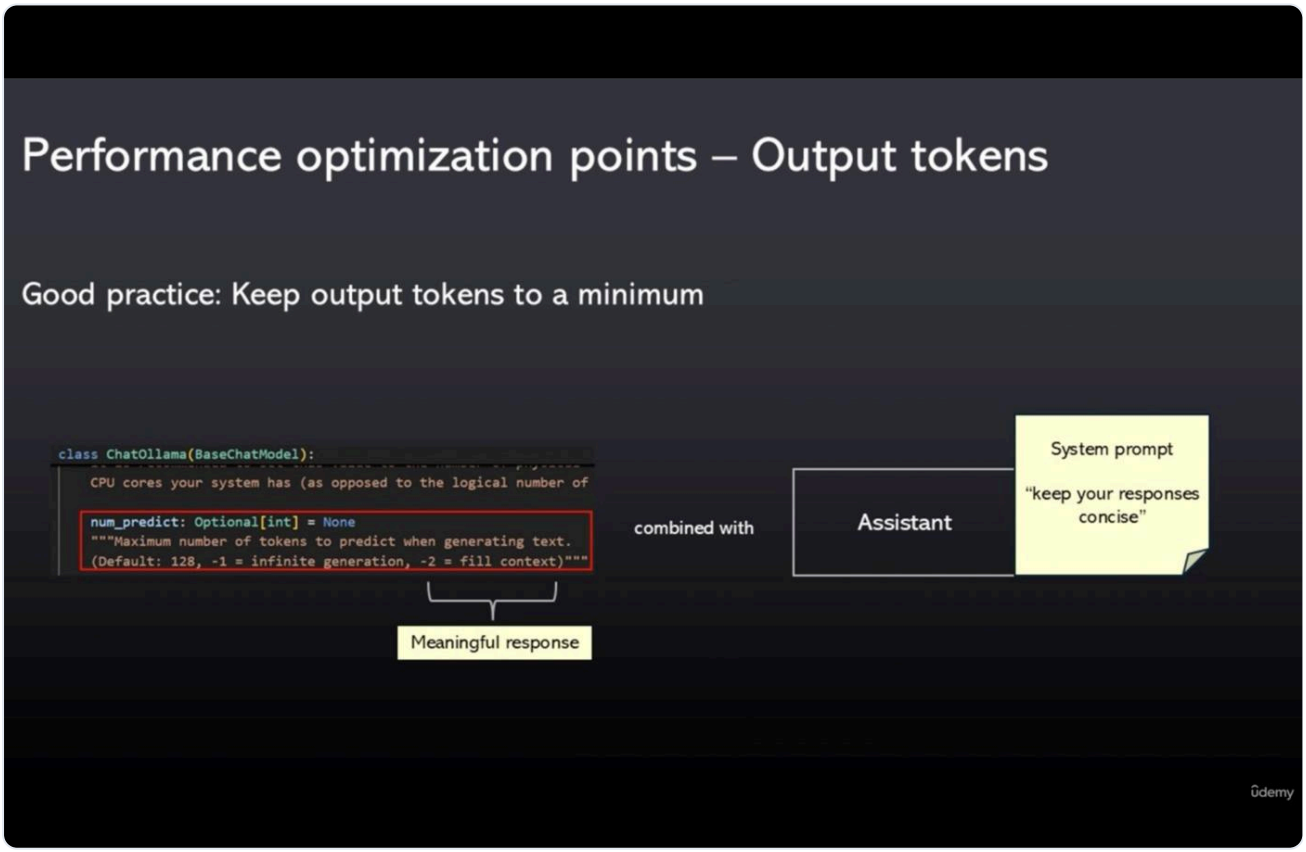


FIGURE 2.29 — PERFORMANCE OPTIMIZATION POINTS

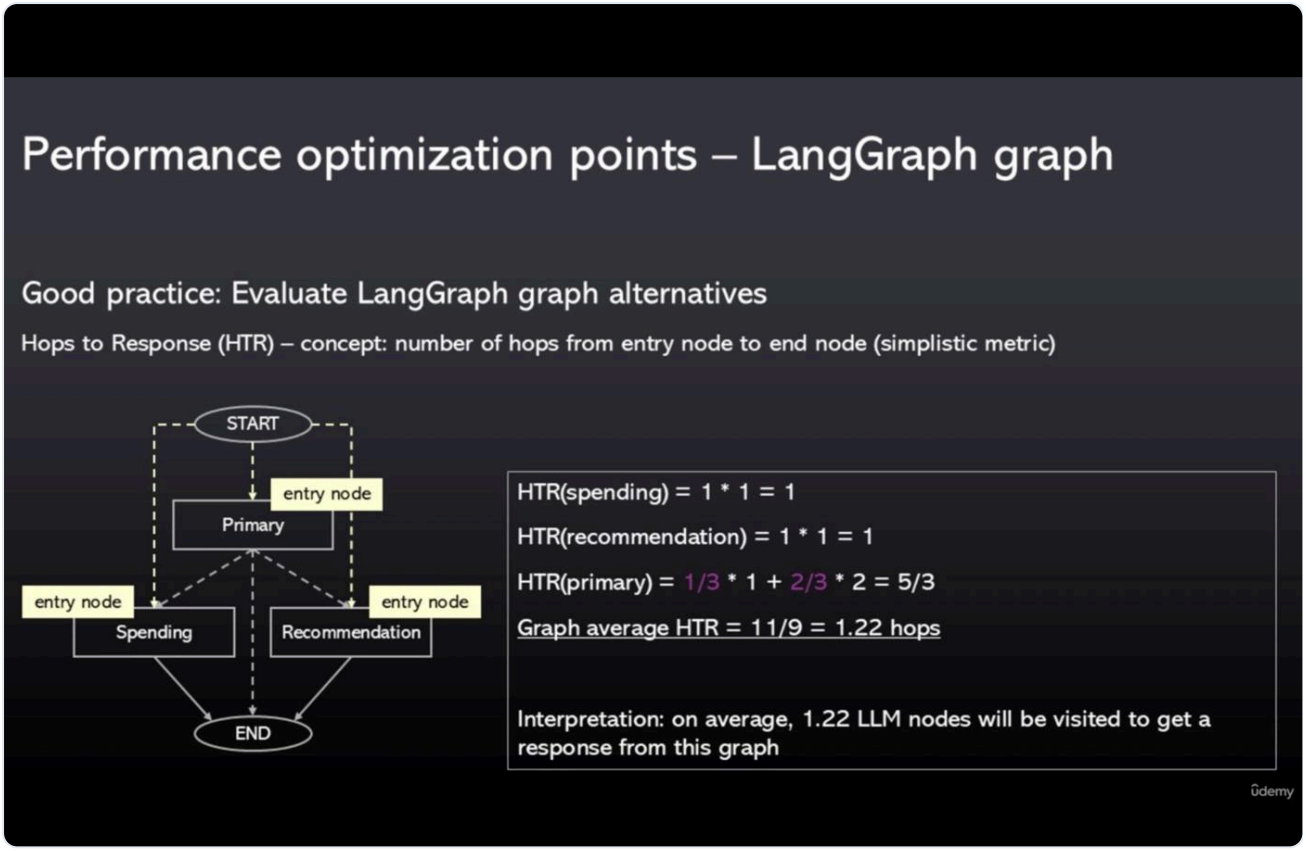


FIGURE 2.30 — PERFORMANCE OPTIMIZATION POINTS

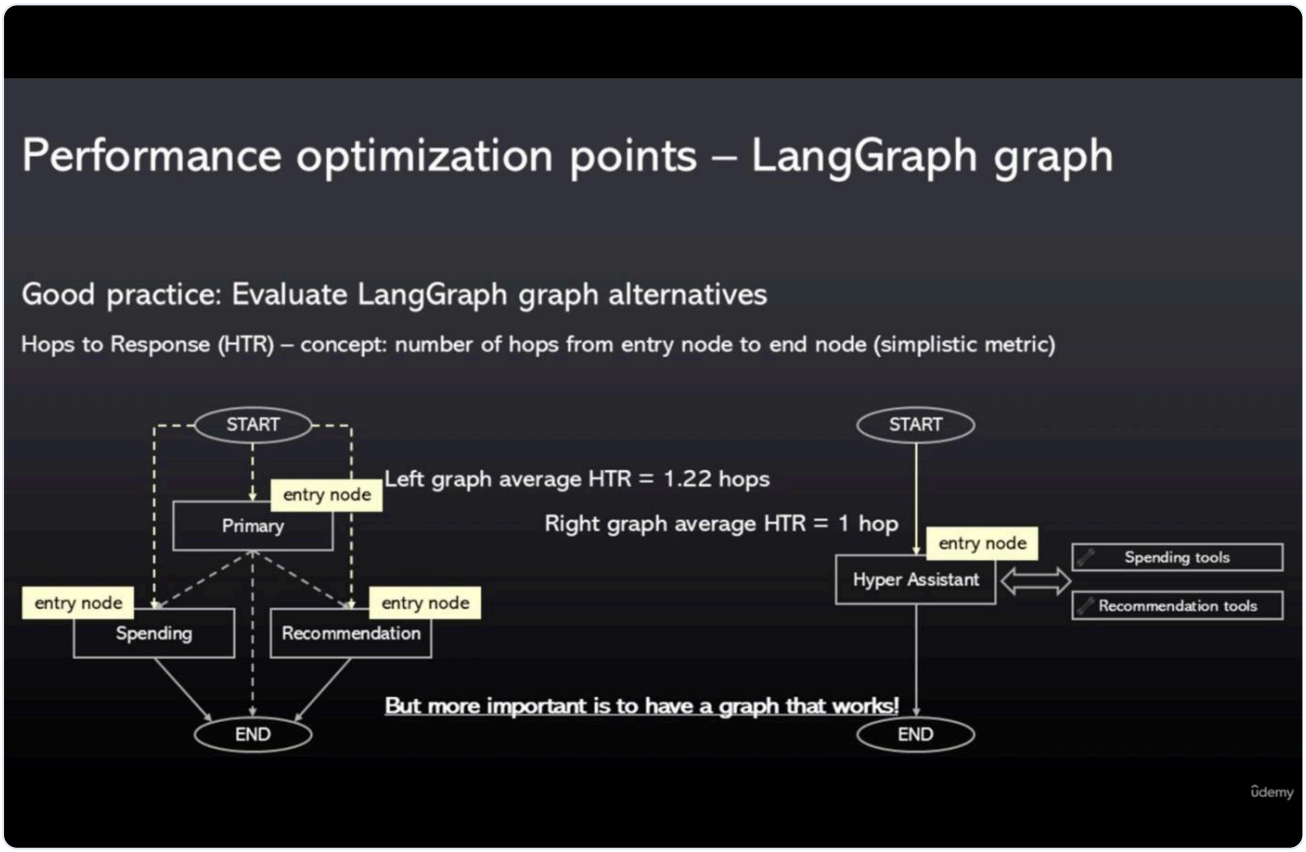


FIGURE 2.31 — PERFORMANCE OPTIMIZATION POINTS

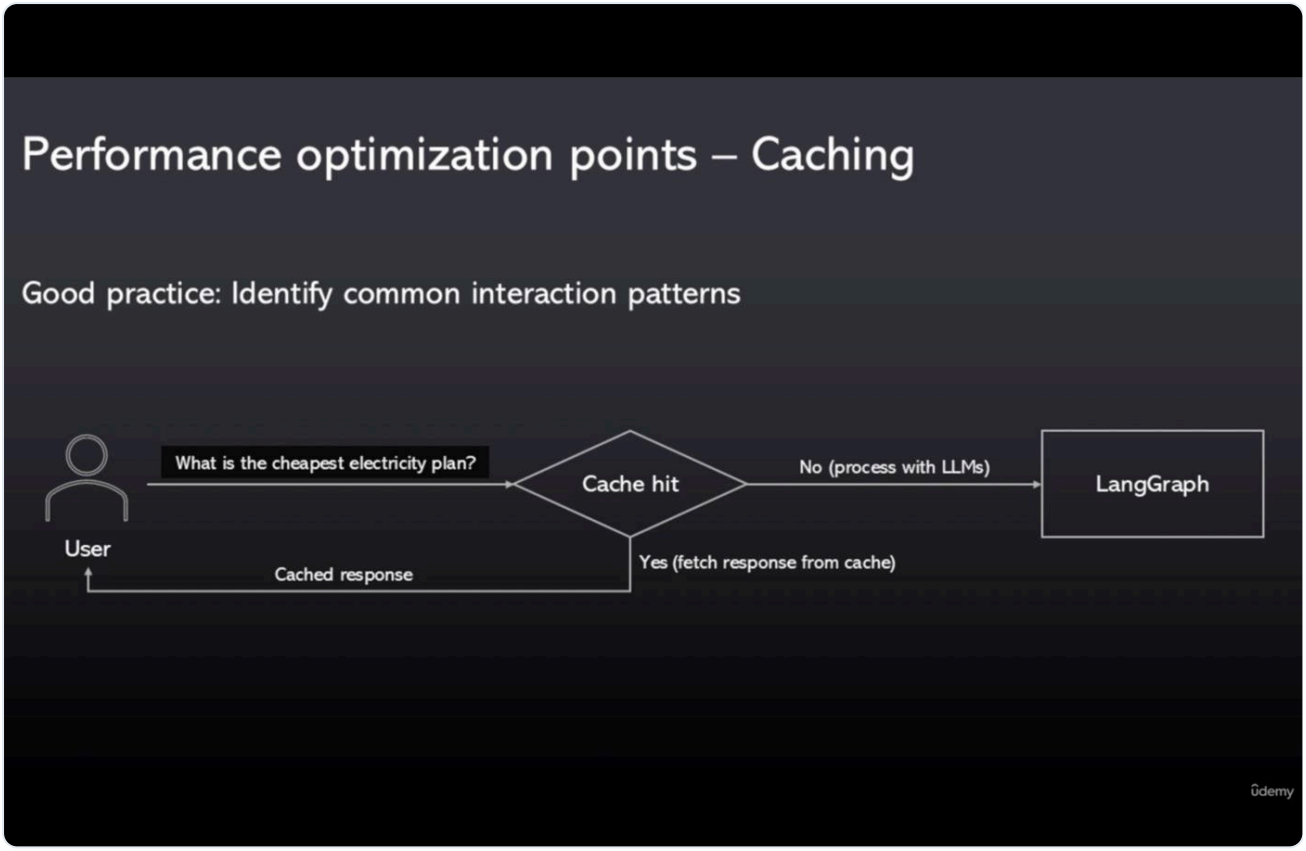


FIGURE 2.32 — PERFORMANCE OPTIMIZATION POINTS

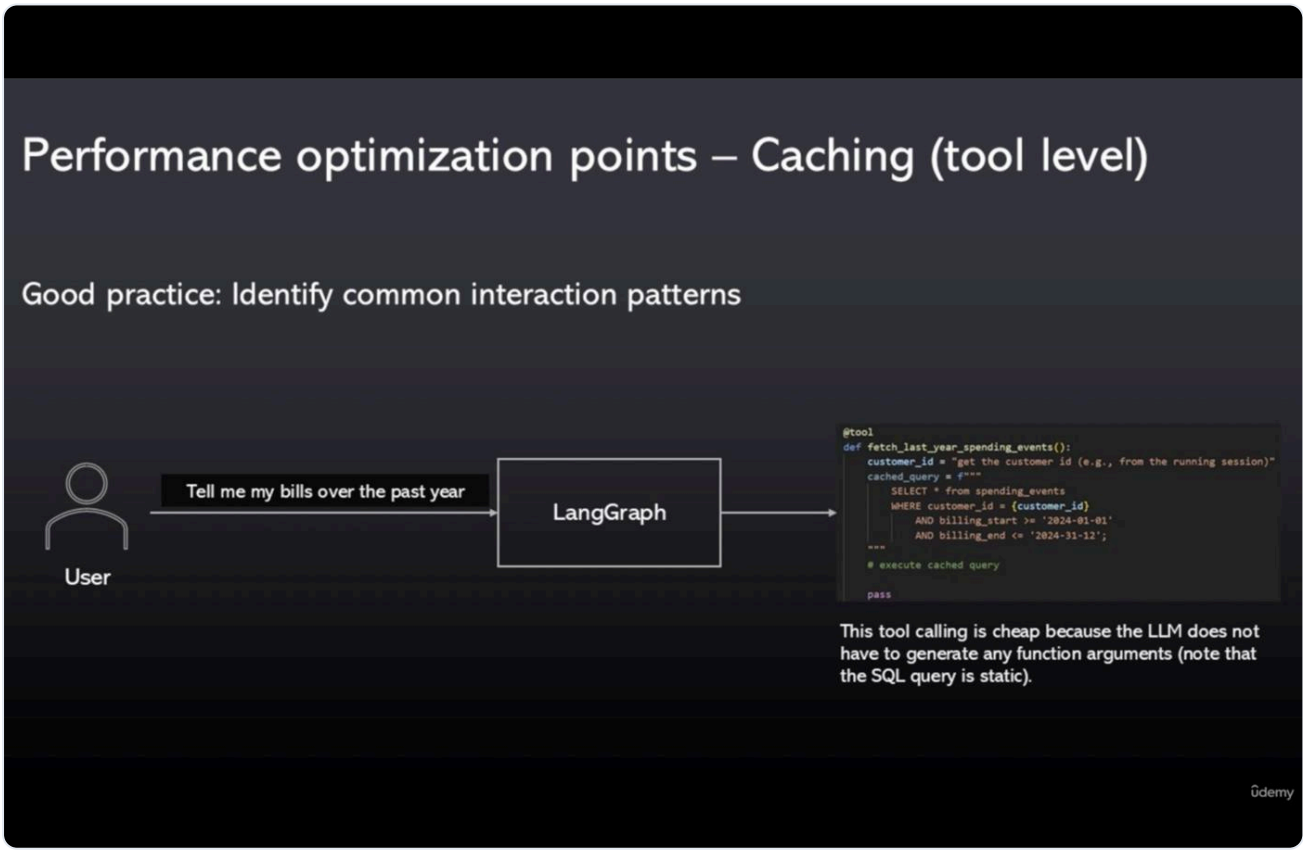


FIGURE 2.33 — PERFORMANCE OPTIMIZATION POINTS

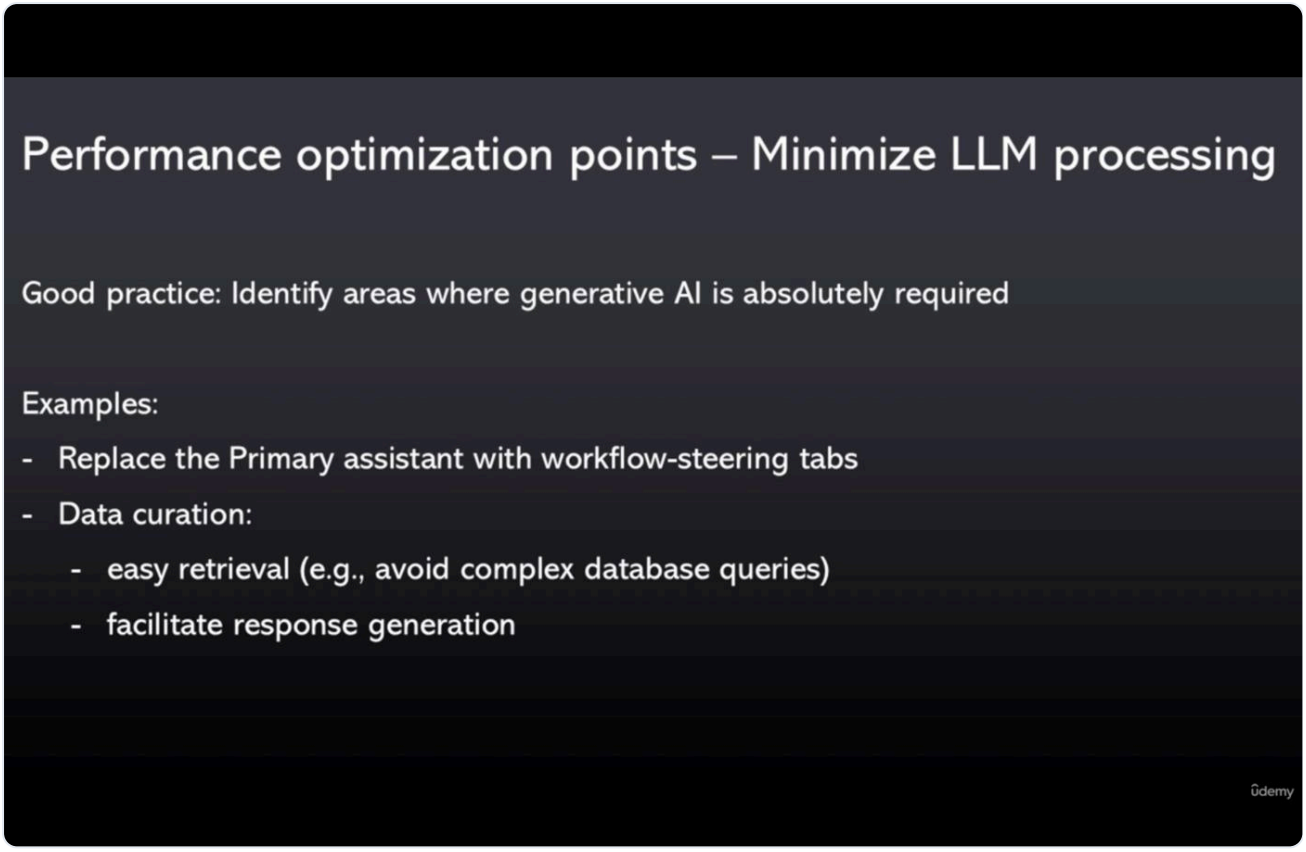


FIGURE 2.34 — PERFORMANCE OPTIMIZATION POINTS

13 · Performance Profiling

Performance profiling is a process that helps us understand the performance behavior of our app w.r.t. varying load levels and resource configurations. Among others, it facilitates:

- Resource optimization
- Load handling
- Cost efficiency

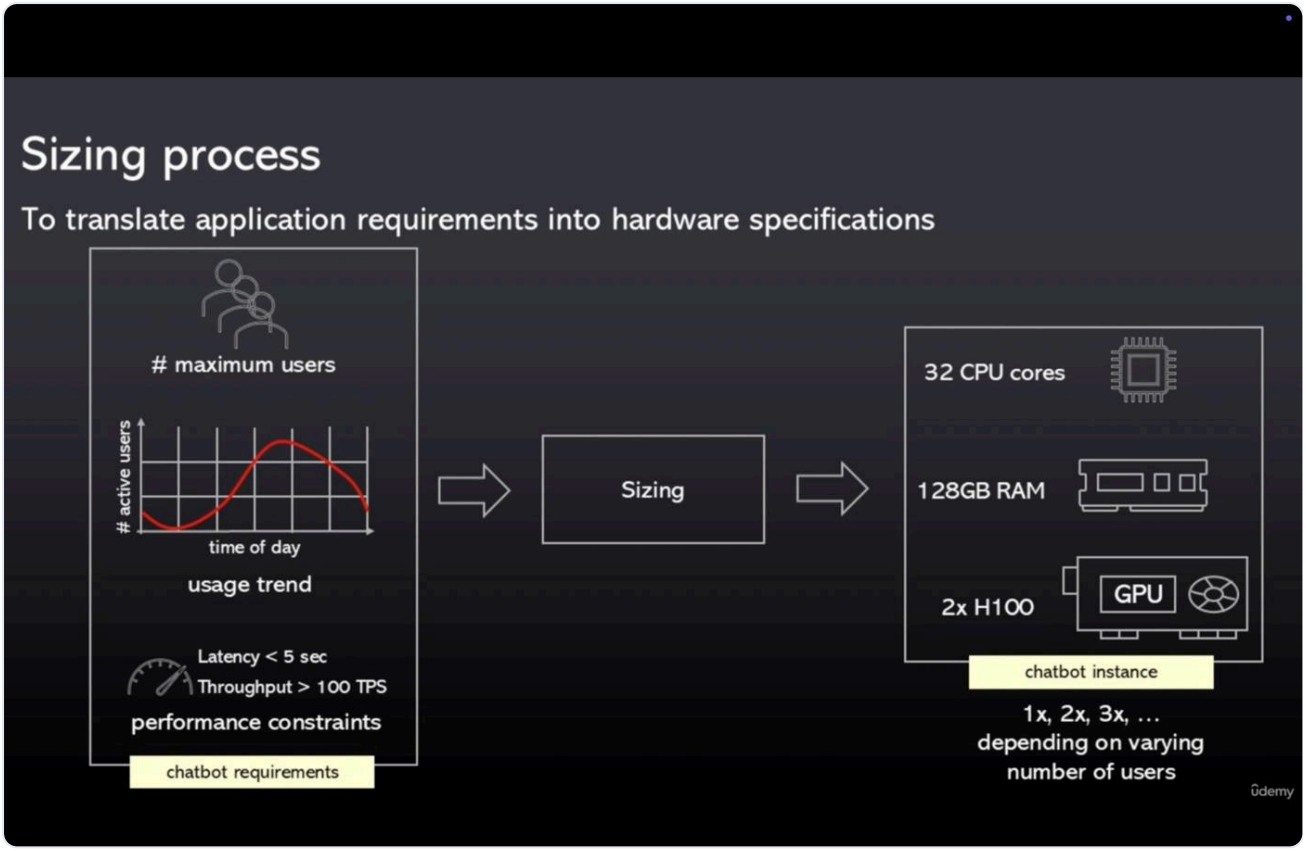


FIGURE 2.35 — PERFORMANCE PROFILING

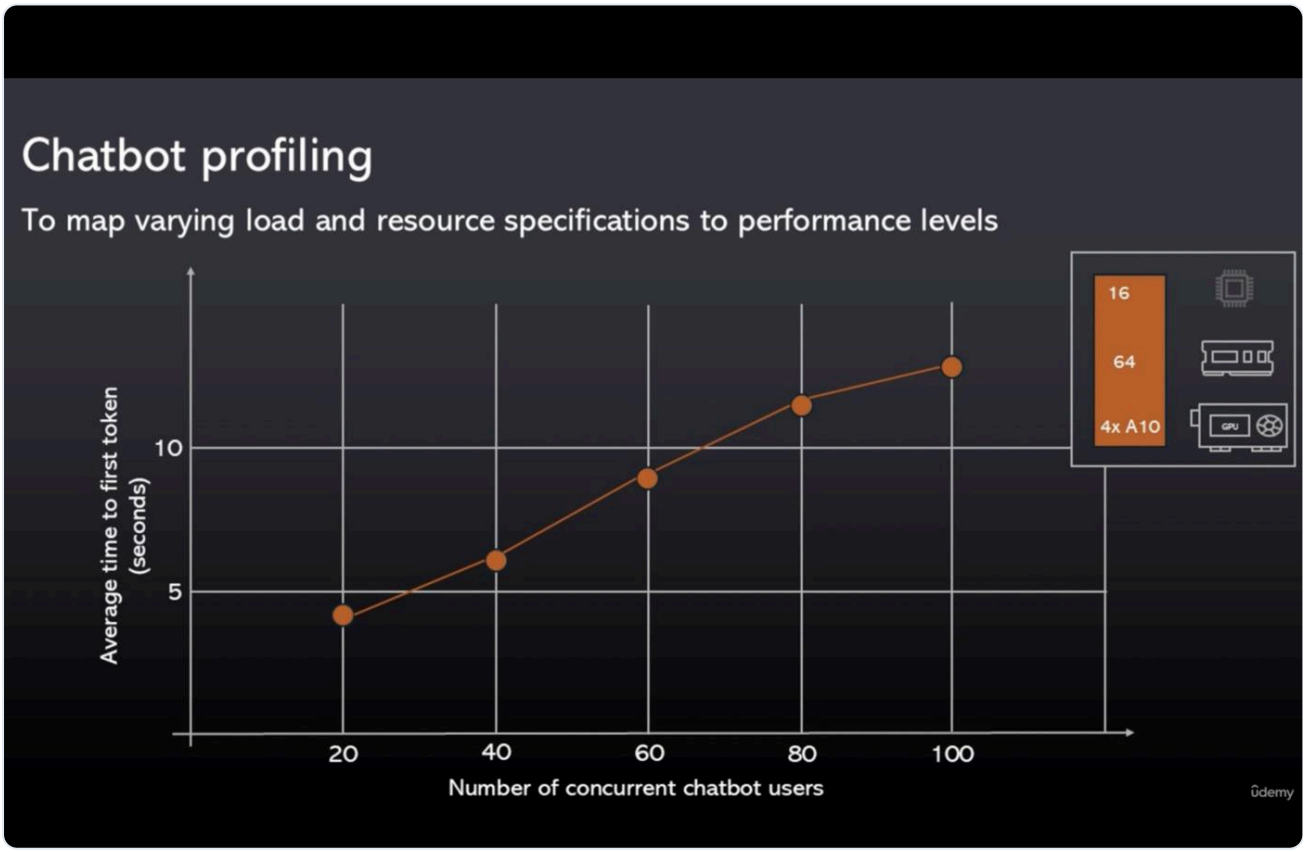


FIGURE 2.36 — PERFORMANCE PROFILING

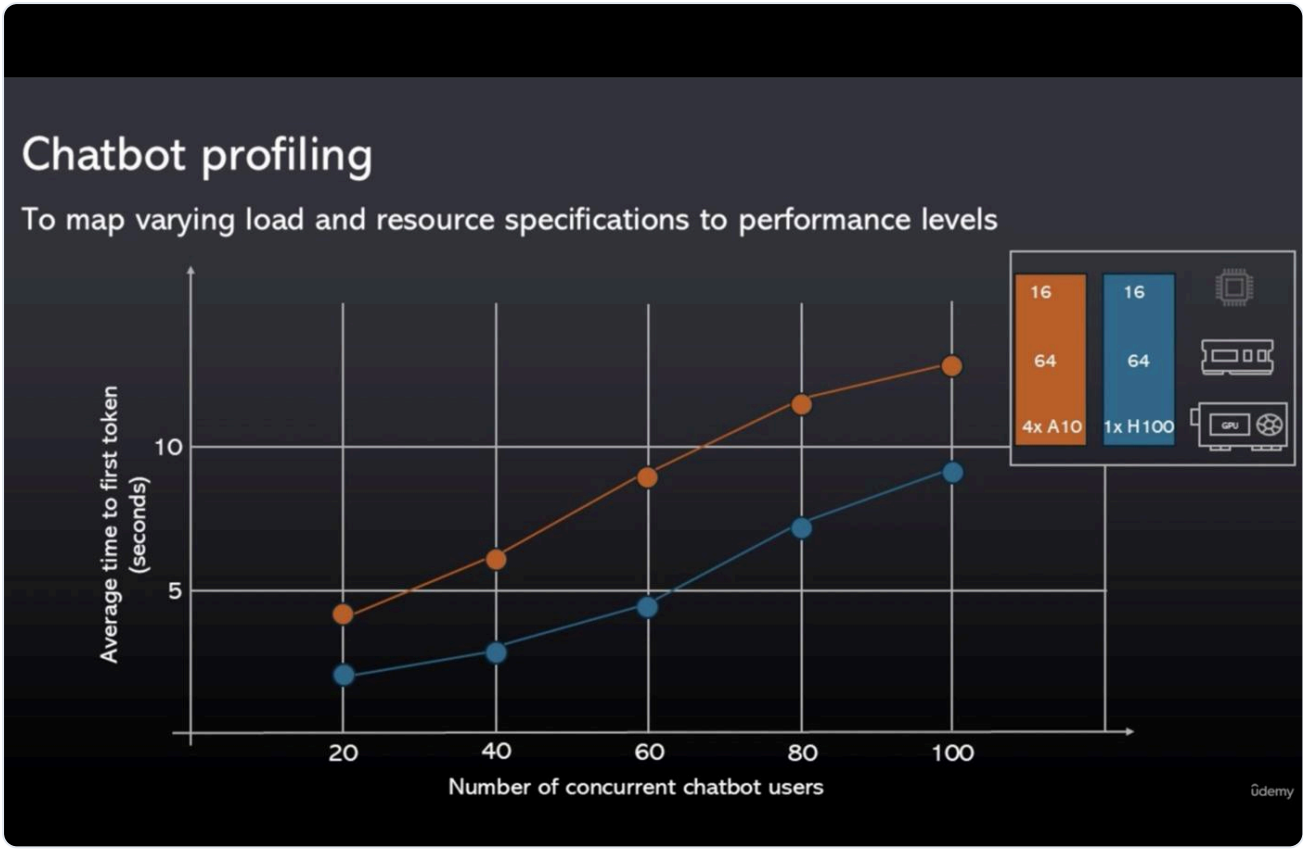


FIGURE 2.37 — PERFORMANCE PROFILING

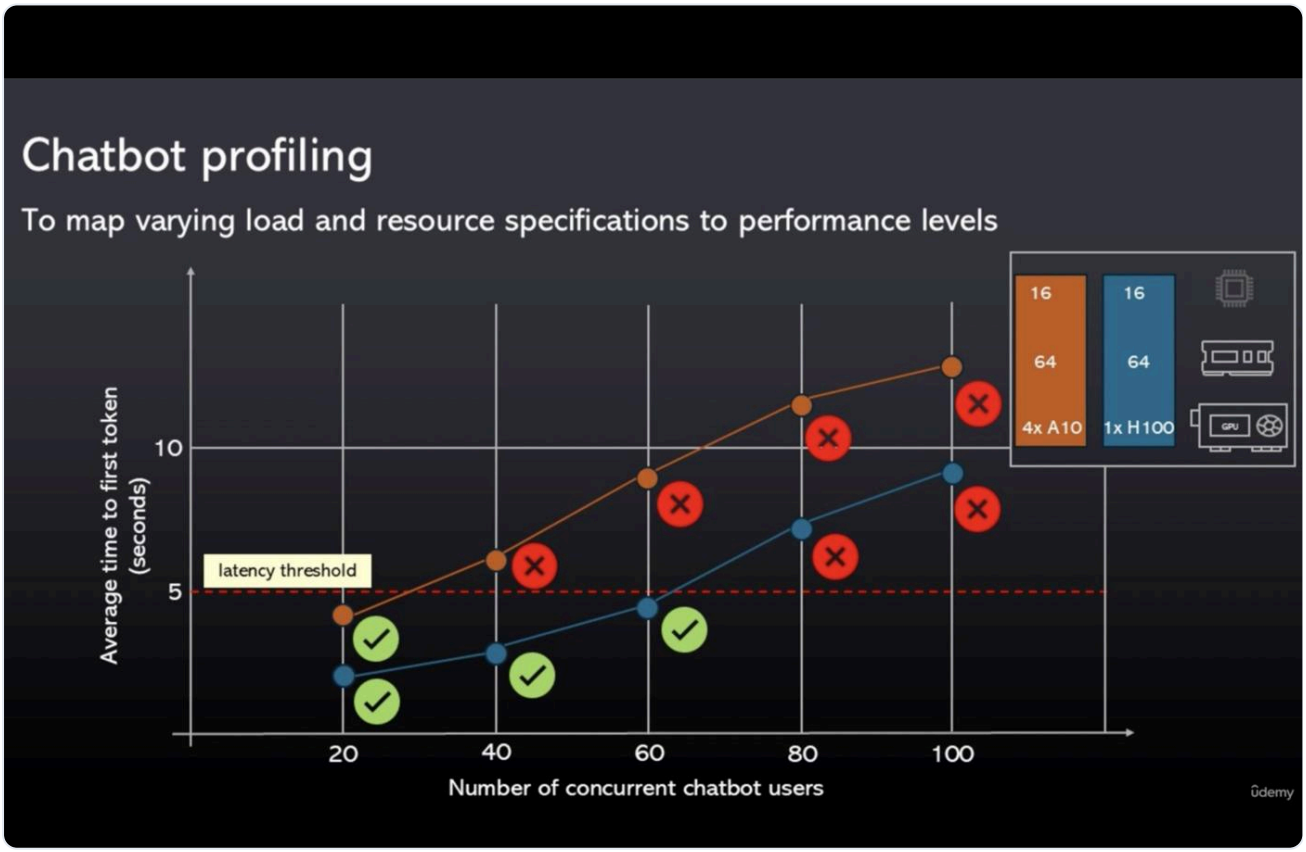


FIGURE 2.38 — PERFORMANCE PROFILING

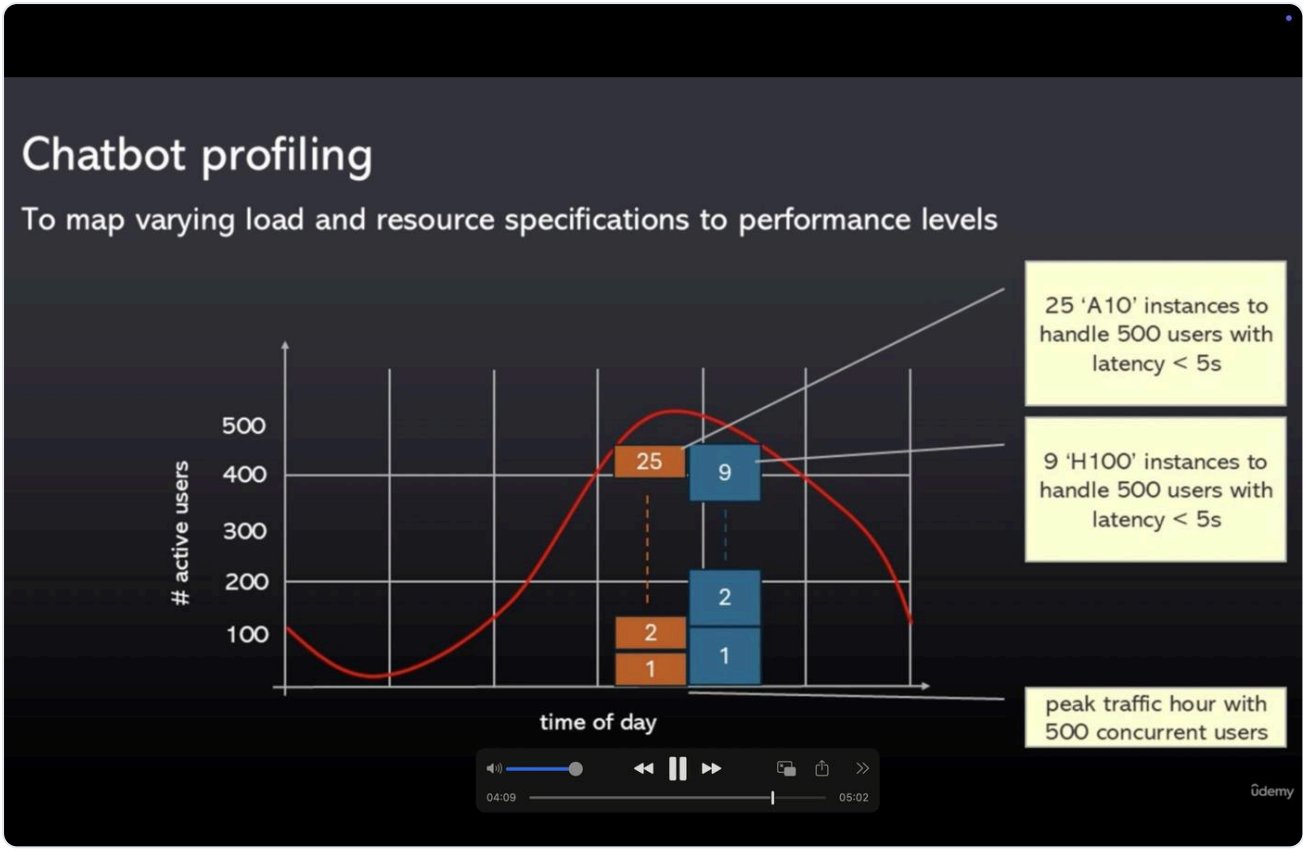


FIGURE 2.39 — PERFORMANCE PROFILING

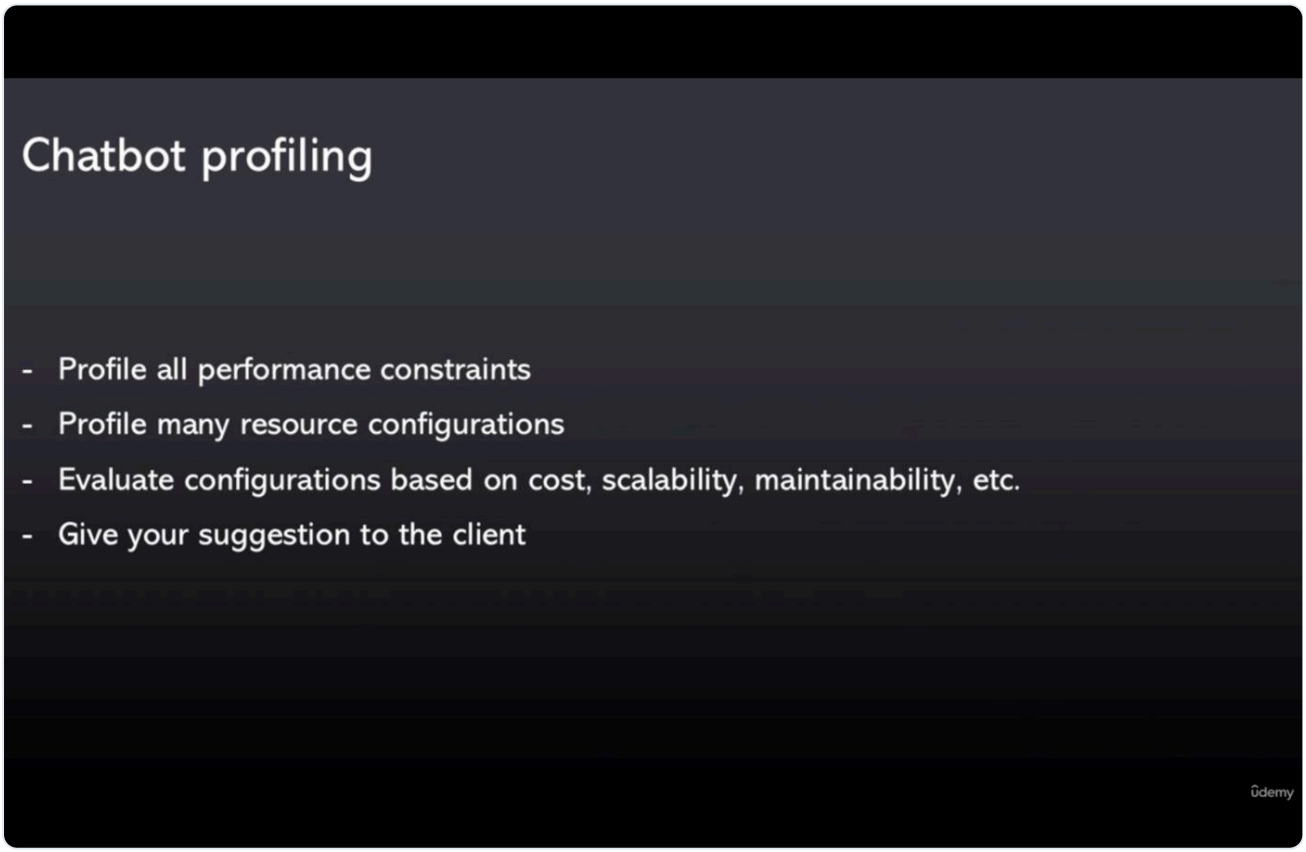


FIGURE 2.40 — PERFORMANCE PROFILING

14 · LLM Infrastructure

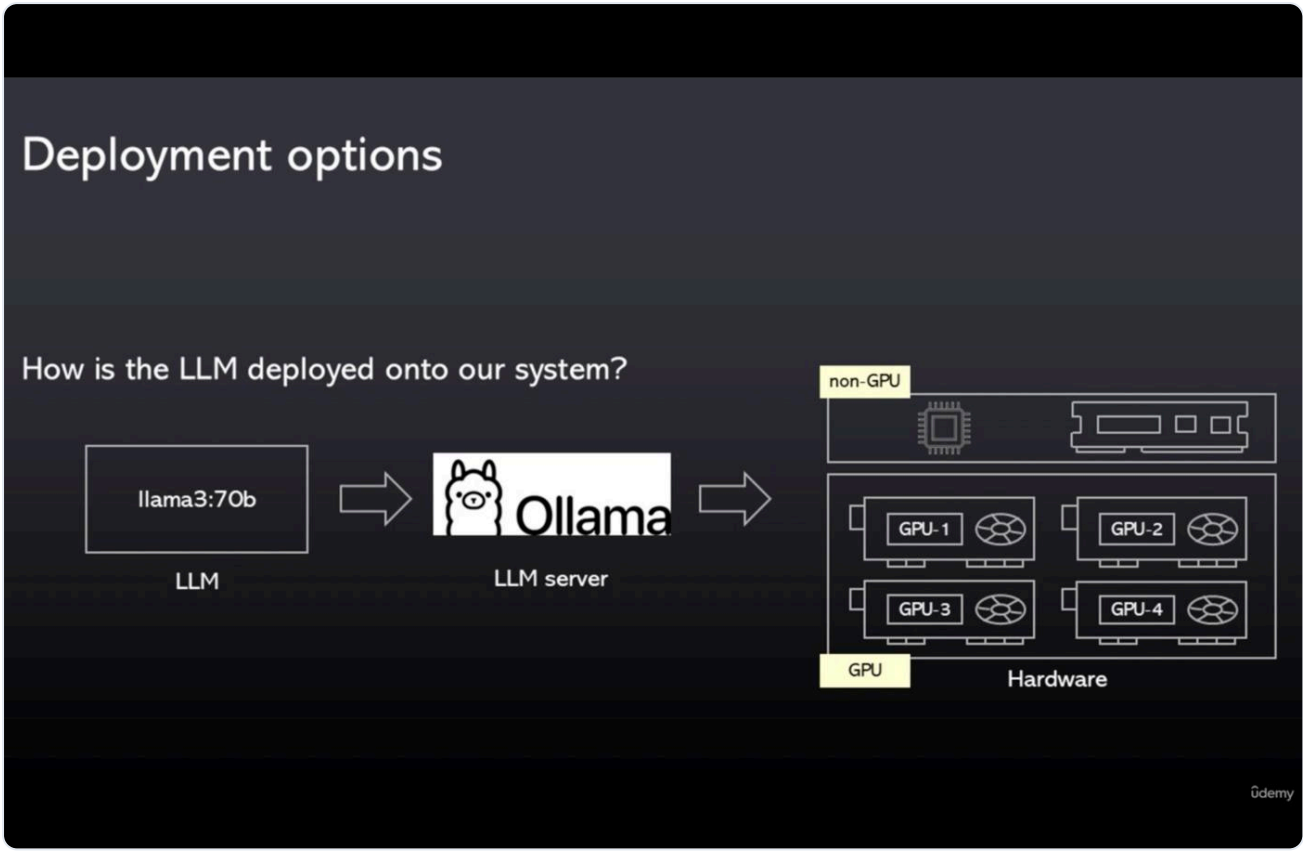


FIGURE 2.41 — LLM INFRASTRUCTURE

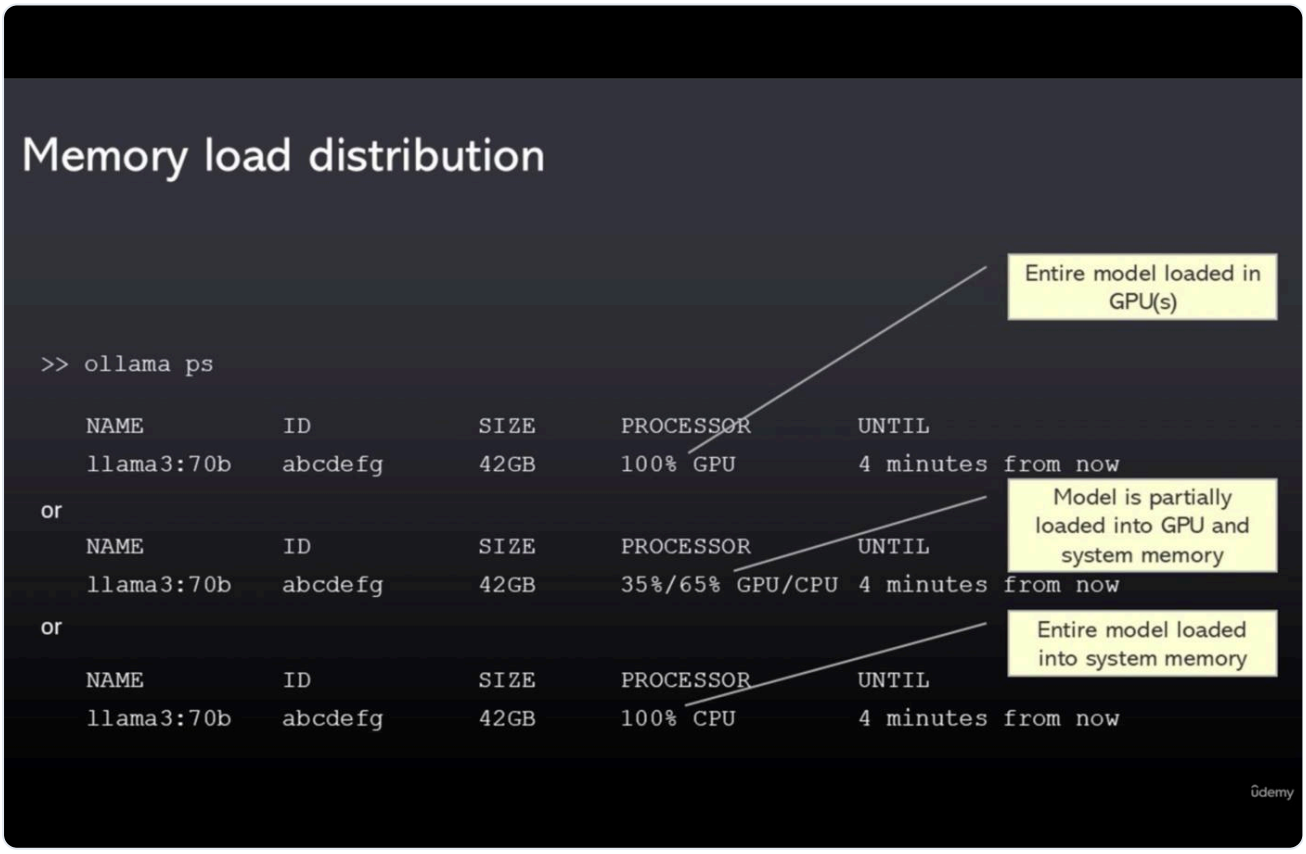


FIGURE 2.42 — LLM INFRASTRUCTURE

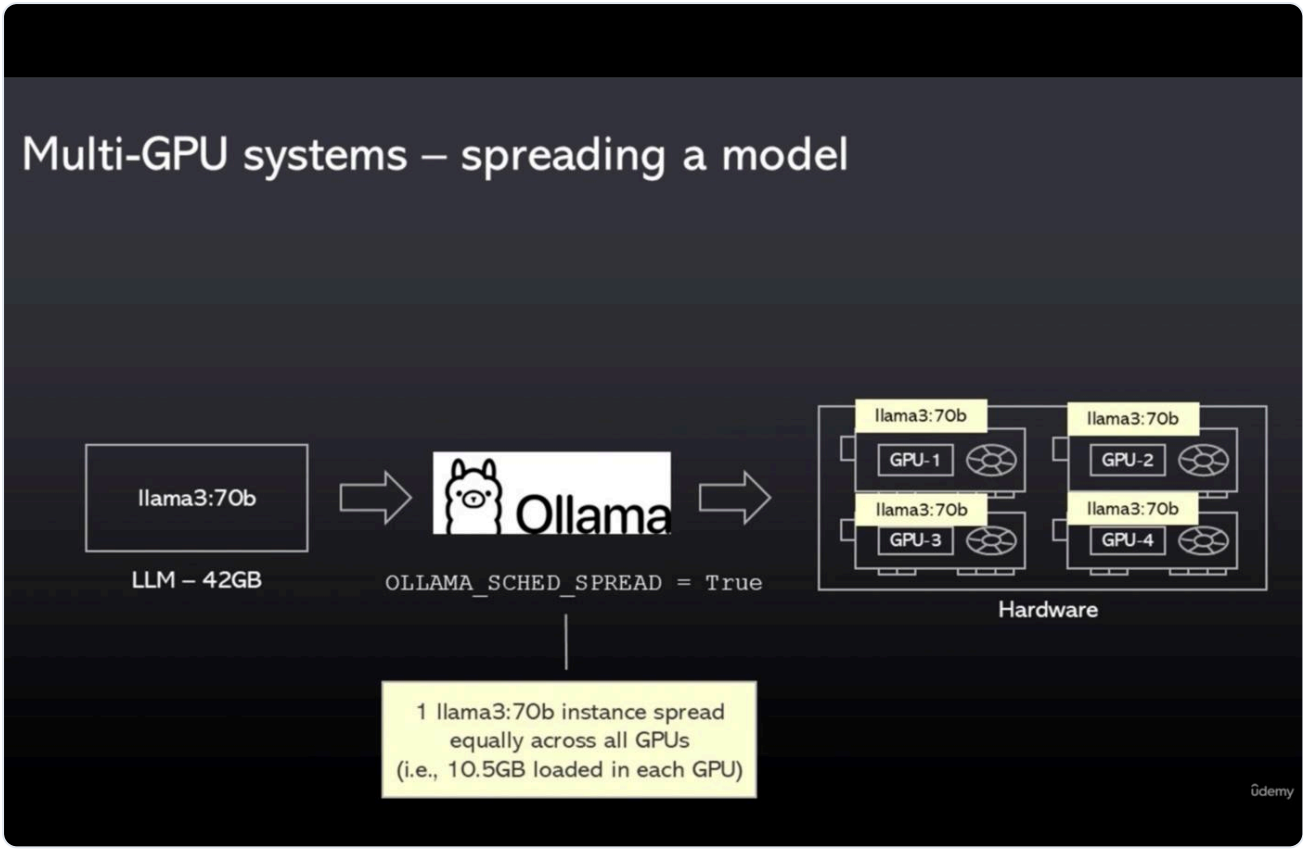


FIGURE 2.43 — LLM INFRASTRUCTURE

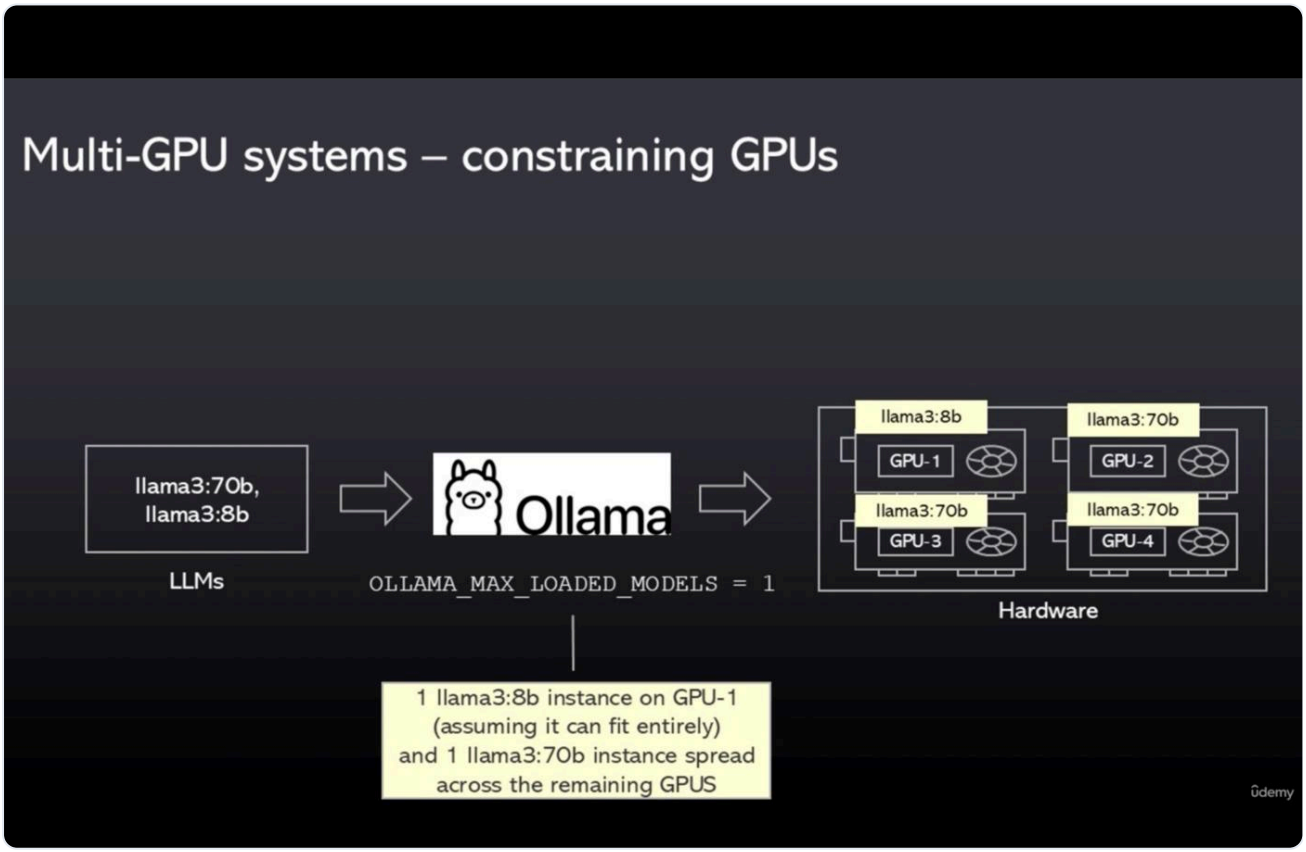


FIGURE 2.44 — LLM INFRASTRUCTURE

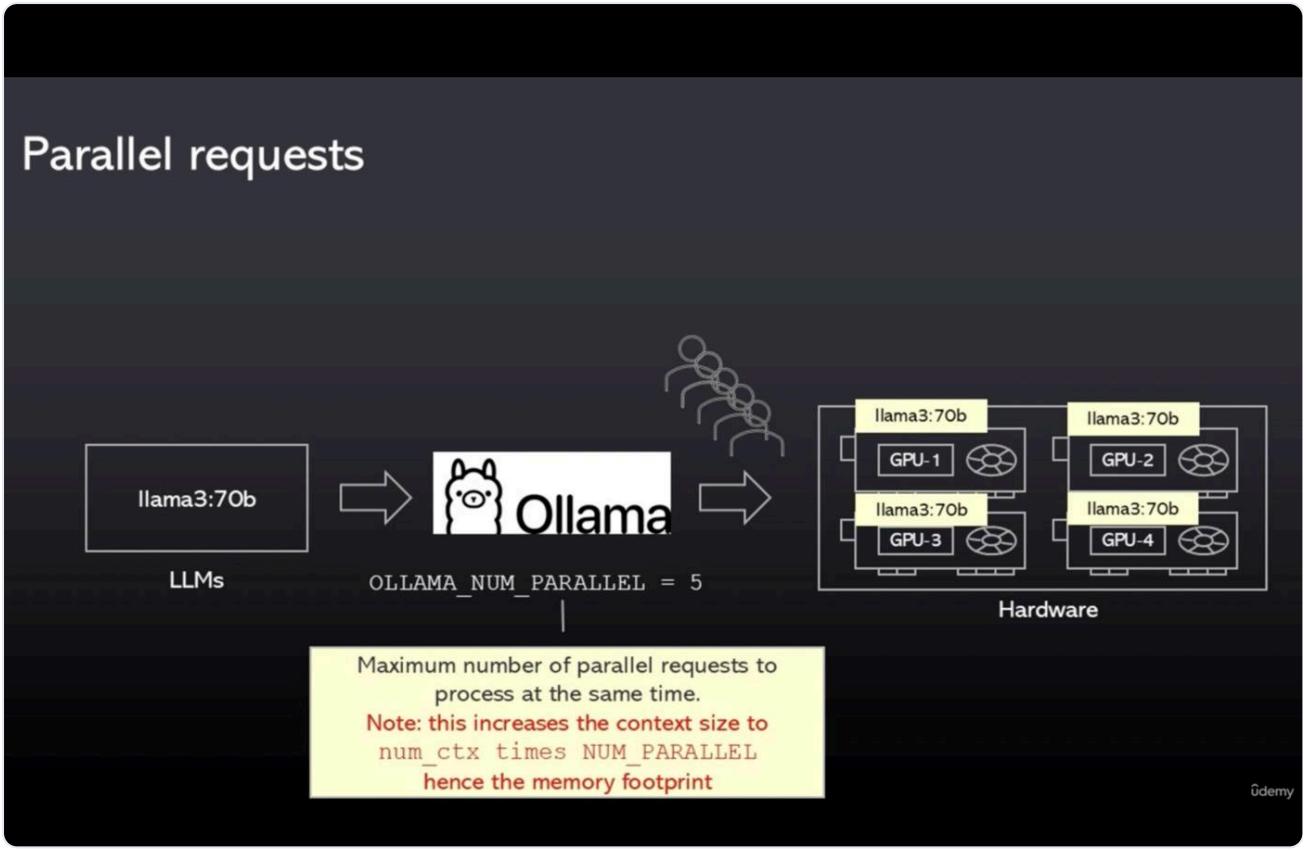


FIGURE 2.45 — LLM INFRASTRUCTURE

15 · Handling long context

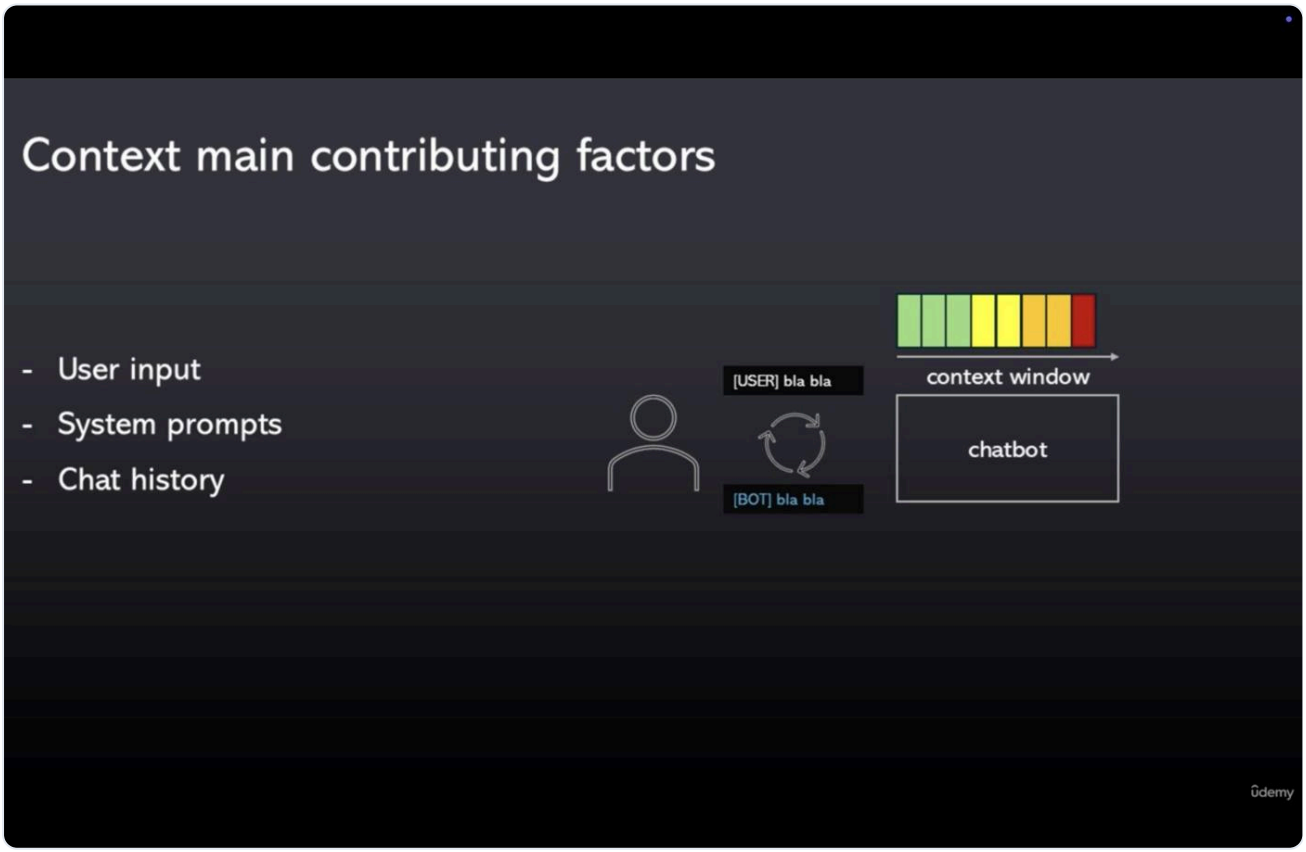


FIGURE 2.46 — HANDLING LONG CONTEXT

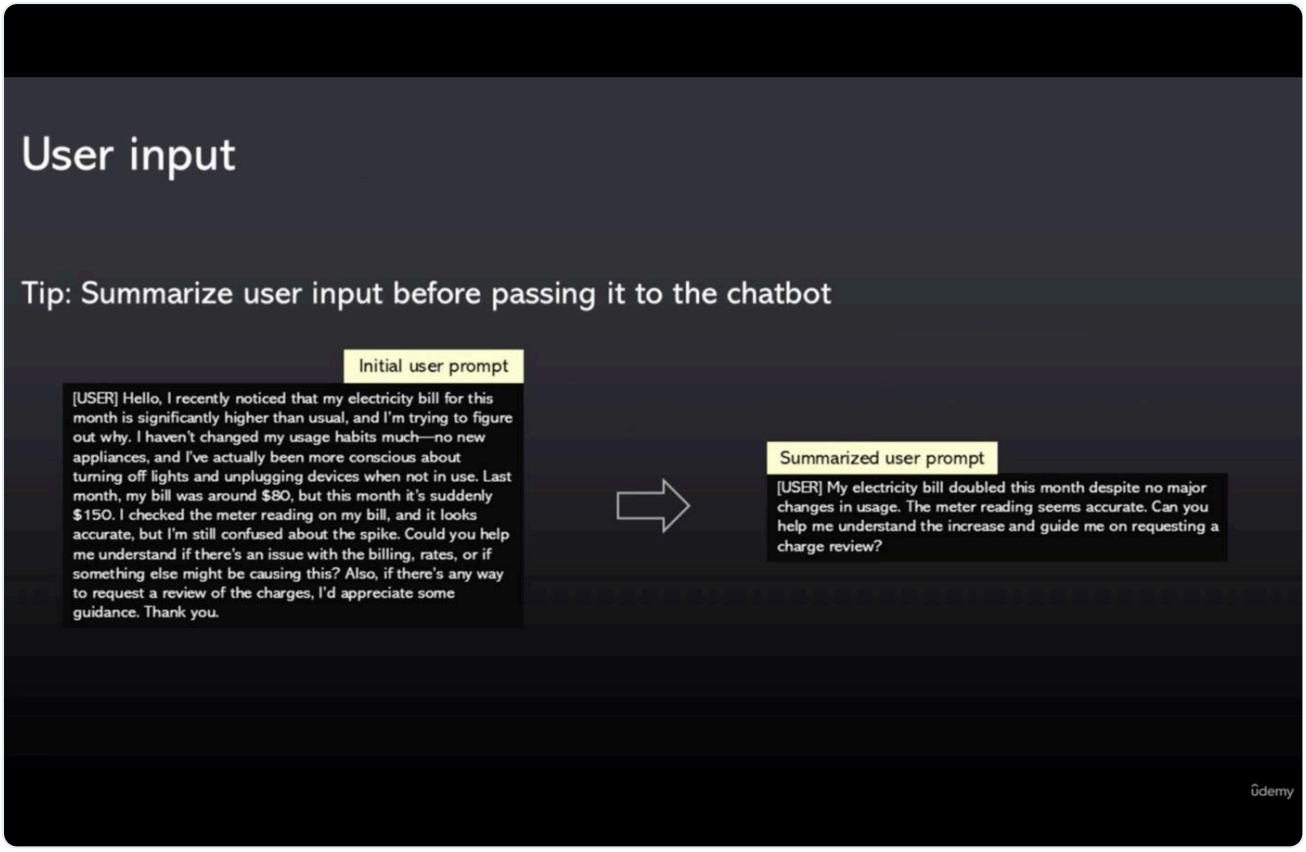


FIGURE 2.47 — HANDLING LONG CONTEXT

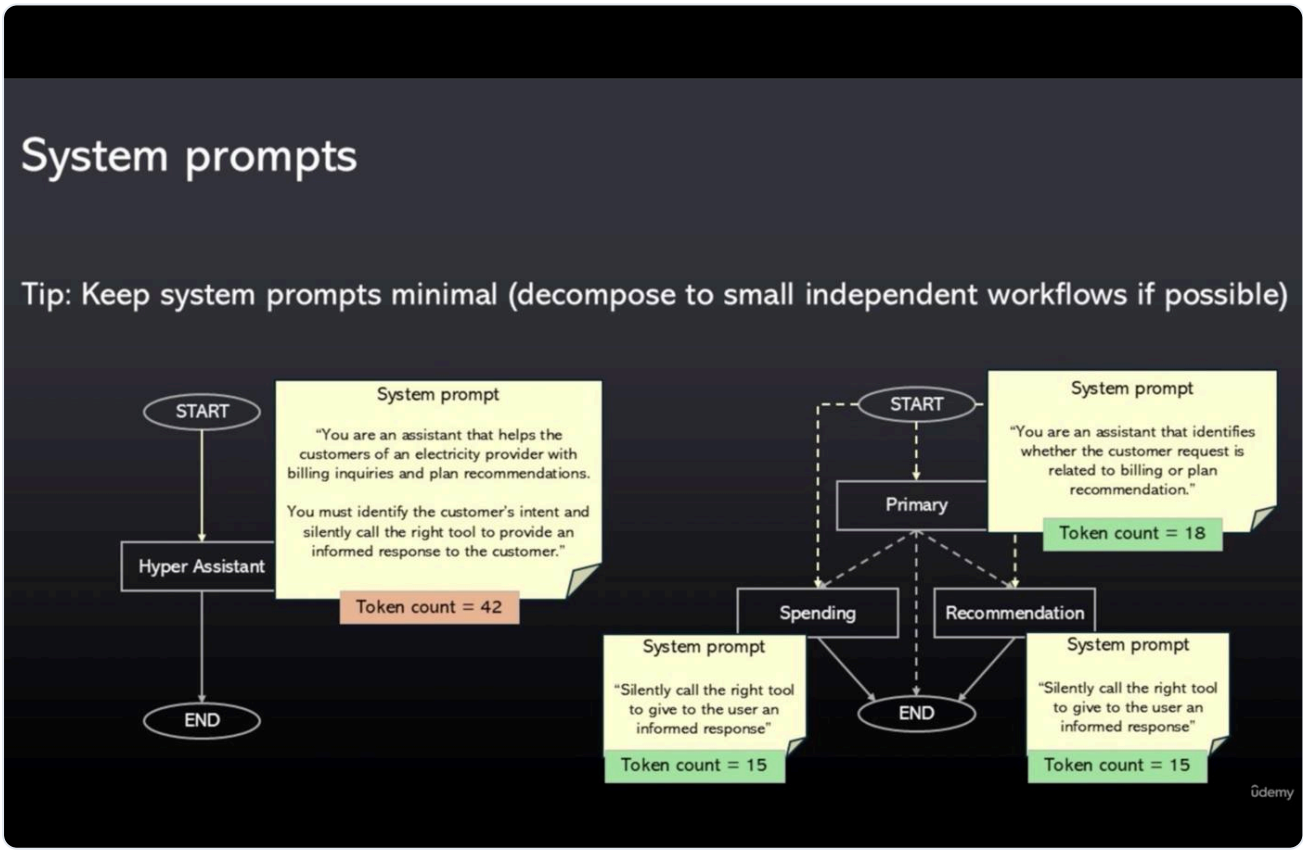


FIGURE 2.48 — HANDLING LONG CONTEXT

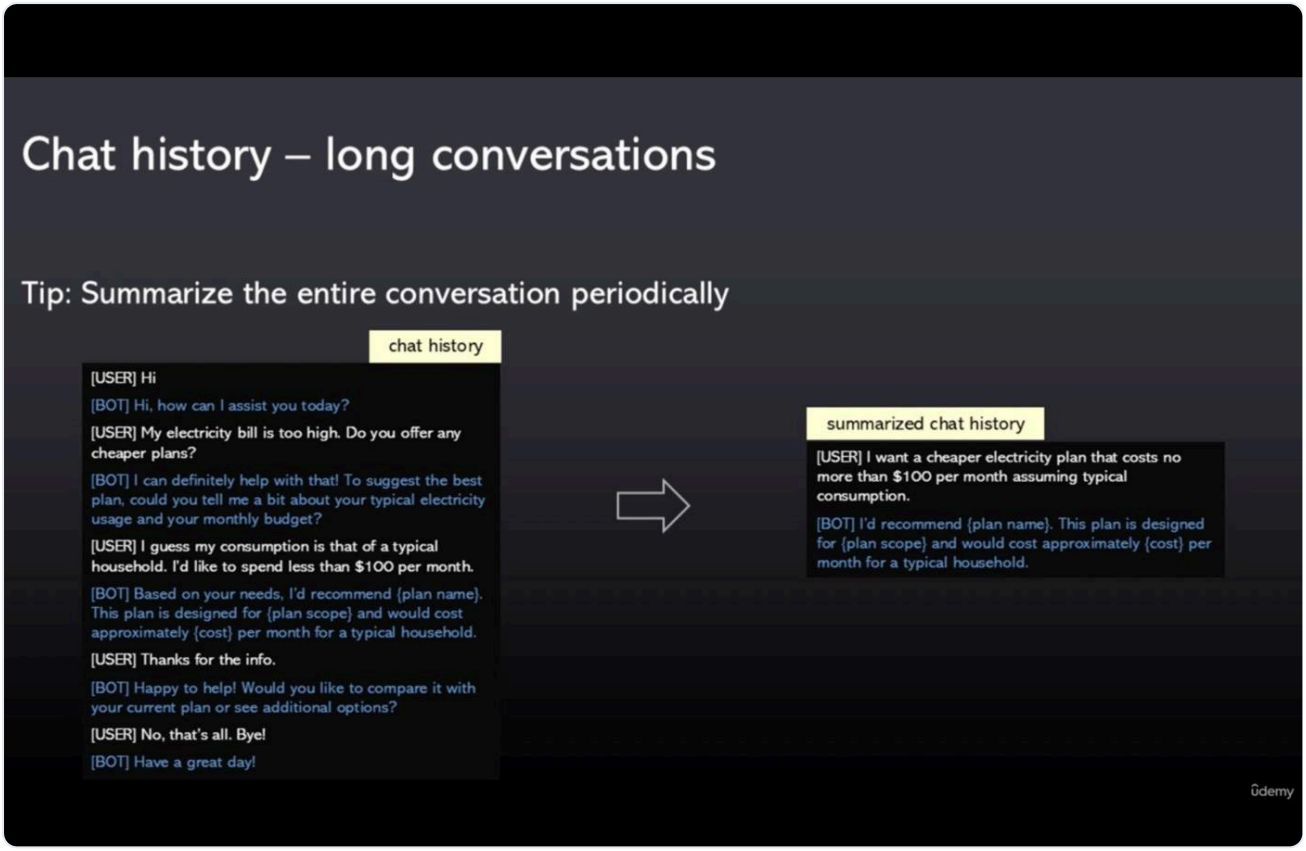


FIGURE 2.49 — HANDLING LONG CONTEXT

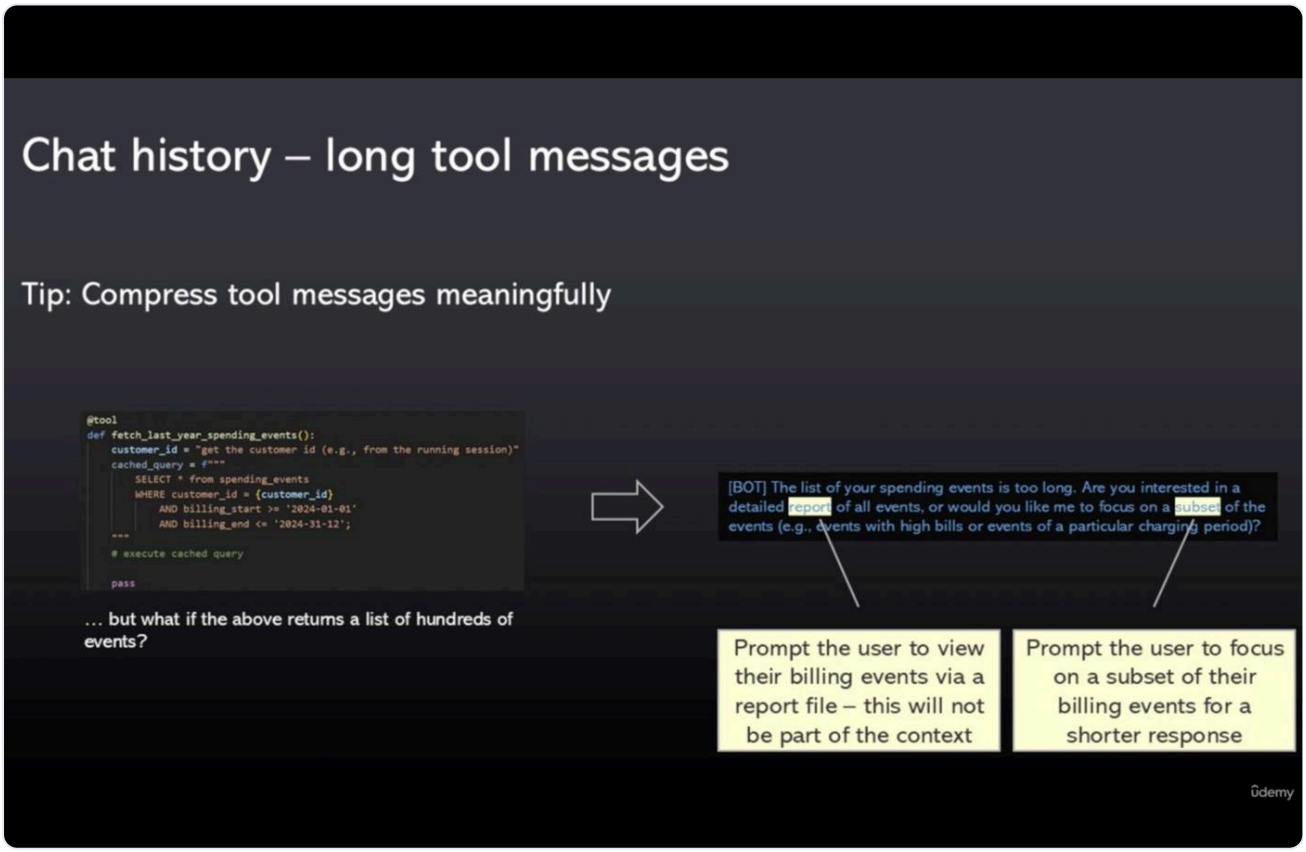


FIGURE 2.50 — HANDLING LONG CONTEXT

16 · Demo to Client

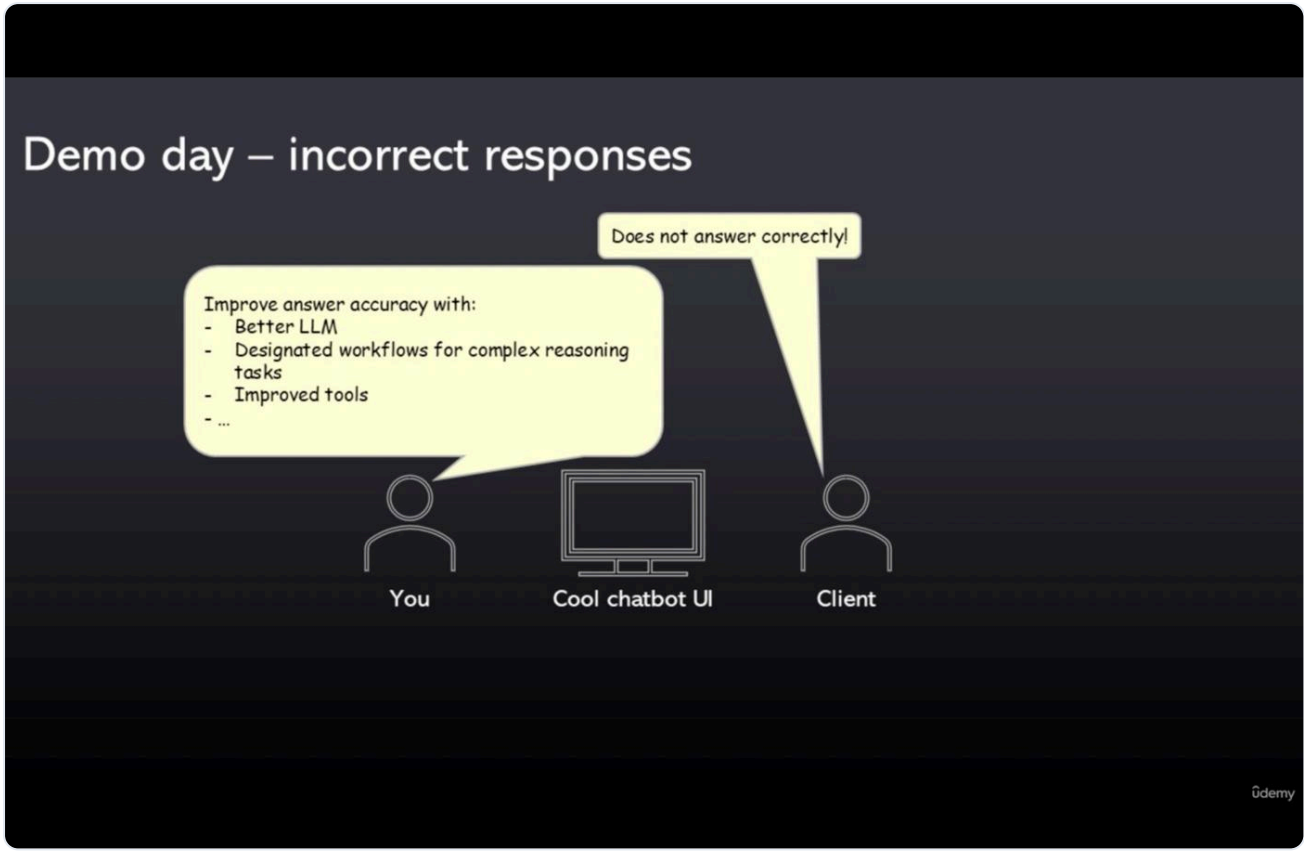


FIGURE 2.51 — DEMO TO CLIENT

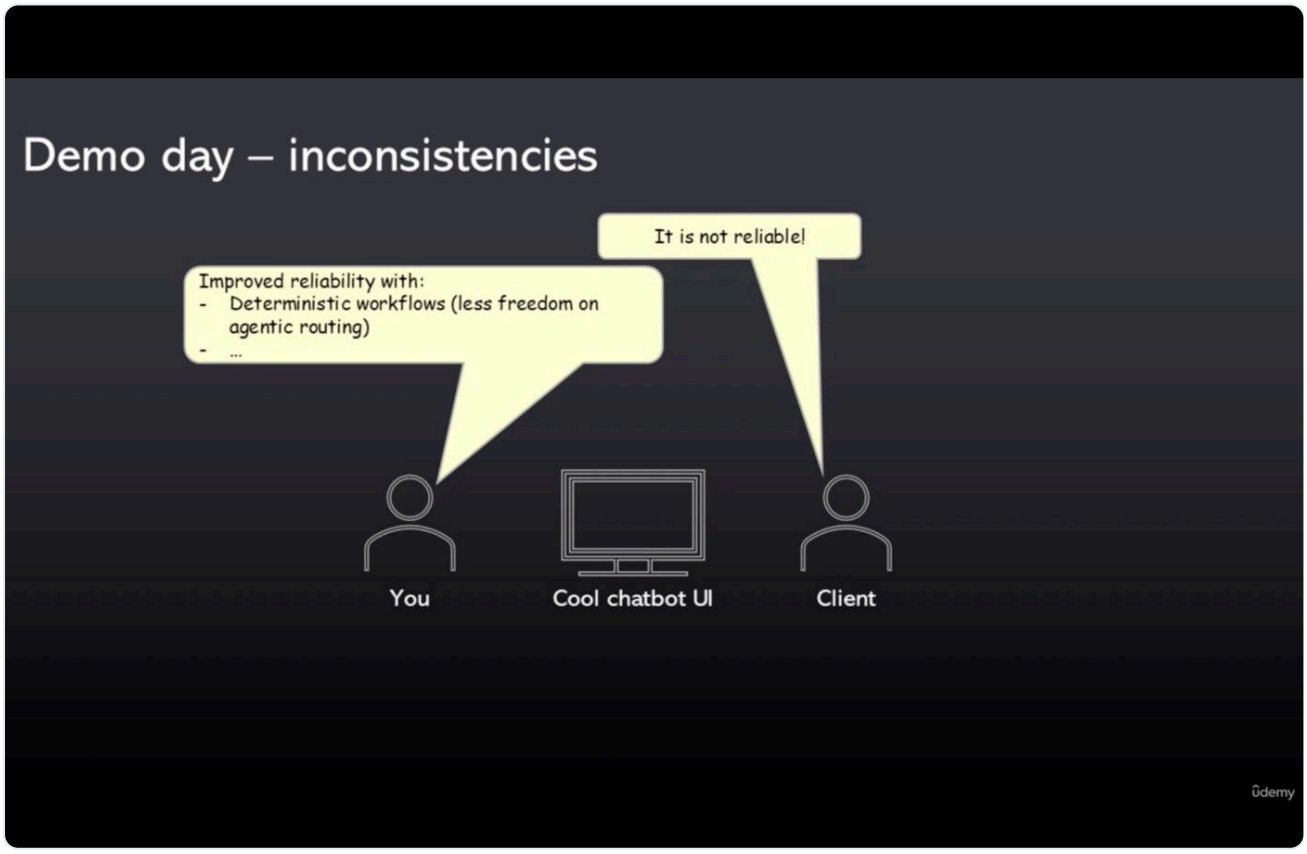


FIGURE 2.52 — DEMO TO CLIENT

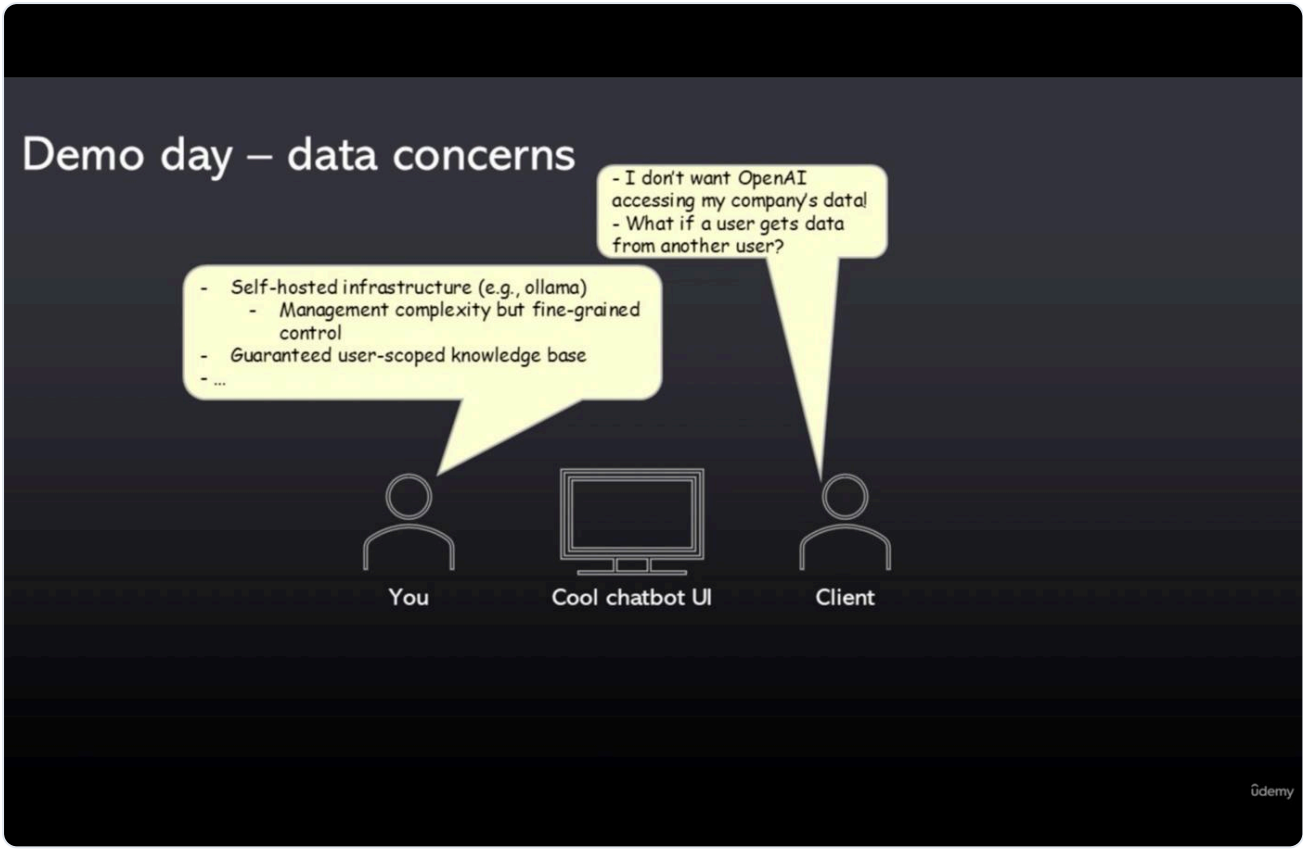


FIGURE 2.53 — DEMO TO CLIENT

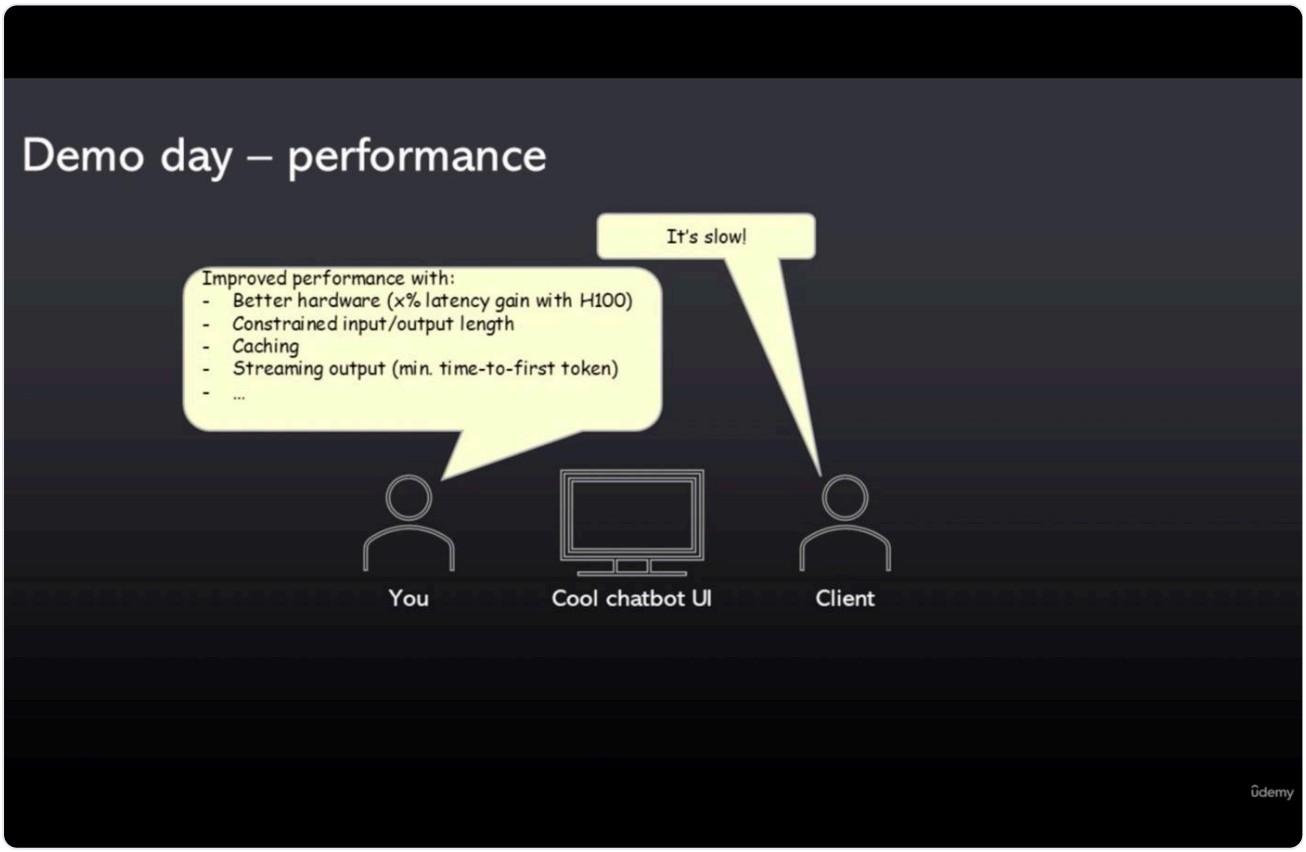


FIGURE 2.54 — DEMO TO CLIENT

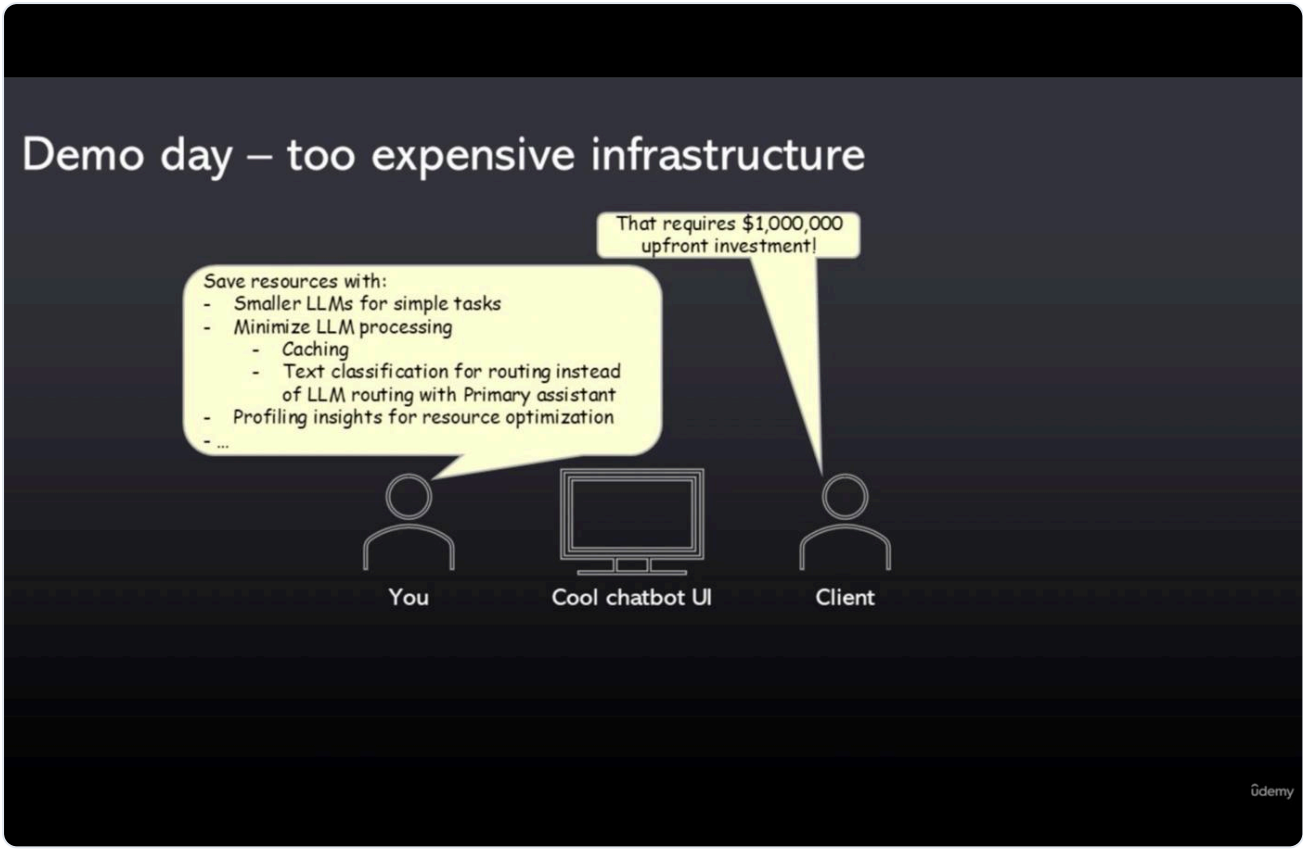


FIGURE 2.55 — DEMO TO CLIENT

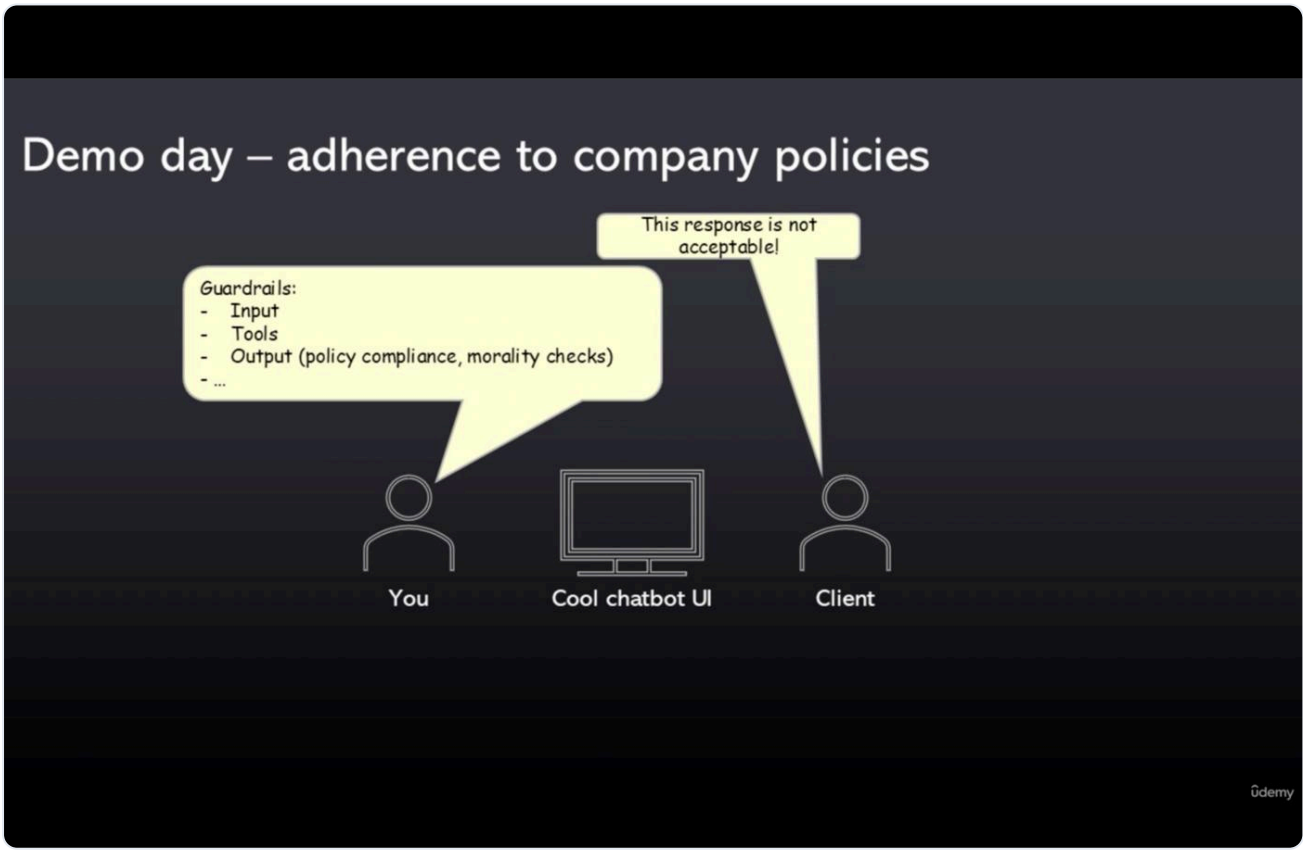


FIGURE 2.56 — DEMO TO CLIENT

17 · Key takeaways

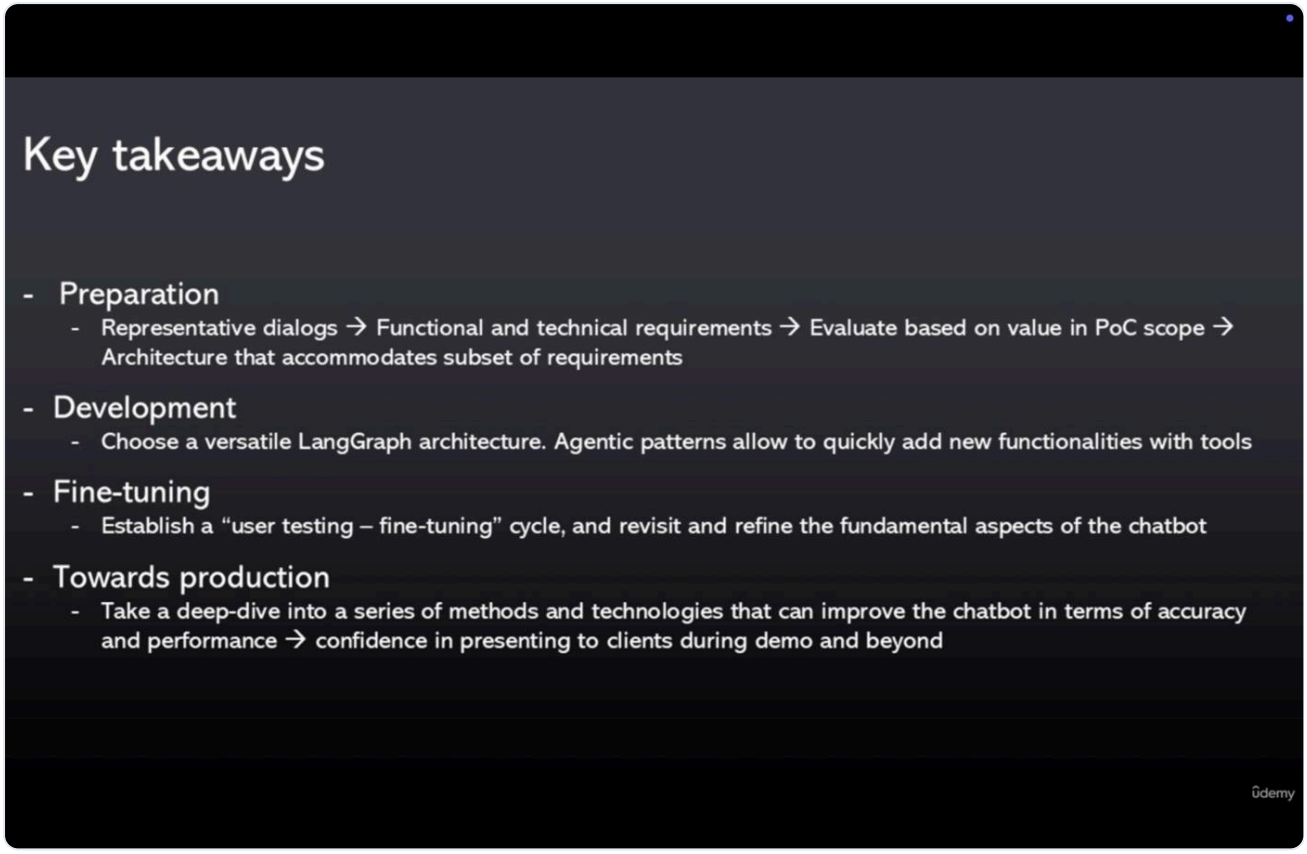


FIGURE 2.57 — KEY TAKEAWAYS